Del Dato a la Decisión: Diseño y Evaluación del Modelo ADA

Fundamentos, Implementación y Resultados de un Modelo de Crédito Tradicional

AUTORES

Diego Lozoya Morales

Arantza Gomez Haro Gamboa

Javier Alejandro Fajardo López

PROFESOR

Rodolfo Slay Ramos

MATERIA

Modelos de Crédito

PROYECTO 1

08 DE MARZO DE 2026

Tabla de Contenido

1. Introducción 6
2. Objetivo del Proyecto 6
   Objetivo General 6
   Objetivos Específicos 7
3. Marco Teórico 7
   Riesgo Crediticio 7
   Modelos de Crédito Tradicionales 8
   FICO Score 8
   Tablero de Puntaje de Crédito 9
   Evaluación del Desempeño de Modelos de Crédito 10
4. Descripción del Modelo ADA 11
   Rango del Puntaje 11
   Clasificación del Riesgo Crediticio 11
   Lógica de Construcción del Puntaje 11
5. Preparación y Limpieza de Datos 12
   Normalización de Valores Faltantes 12
   Corrección de Formatos 12
   Validación 12
   Imputación Estructurada 12
   Transformación de Variables 13
6. Benchmark 13
   Regresión Logística Simple 13
   Resultados del Benchmark 13
7. Selección de Variables 14
   Objetivo de la Selección de Variables 14
   Metodología XGBoost 14
   Interpretación de Variables mediante Valores SHAP 15
   Resultados de la Selección de Variables 15
8. Asignación de Signos 16
   Outstanding Debt 16
      Descripción de la Variable 16
      Relevancia para el Riesgo Crediticio 16
      Explicación del Signo 16
   Interest Rate 16

Descripción de la Variable 16
Relevancia para el Riesgo Crediticio 17
Explicación del Signo 17
Delay from Due Date 17
Descripción de la Variable 17
Relevancia para el Riesgo Crediticio 17
Explicación del Signo 17
Number of Credit Cards 17
Descripción de la Variable 17
Relevancia para el Riesgo Crediticio 18
Explicación del Signo 18
Credit Mix Standard 18
Descripción de la Variable 18
Relevancia para el Riesgo Crediticio 18
Explicación del Signo 18
Change Credit Limit 18
Descripción de la Variable 18
Relevancia para el Riesgo Crediticio 19
Explicación del Signo 19
Credit Mix Good 19
Descripción de la Variable 19
Relevancia para el Riesgo Crediticio 19
Explicación del Signo 19
Payment of Min Amount Yes 19
Descripción de la Variable 19
Relevancia para el Riesgo Crediticio 20
Explicación del Signo 20
High Spent 20
Descripción de la Variable 20
Relevancia para el Riesgo Crediticio 20
Explicación del Signo 20
9. Estimación de Pesos 20
Metodología para la Estimación de Pesos 20
Pesos Obtenidos 21
Interpretación de las Variables 21

Credit Mix Good 21
Payment of Minimum Amount Yes 21
Credit Mix Standard 21
High Spent 22
Pesos Más Bajos 22
10. Puntaje de crédito 22
11. Tablero de Puntaje de Crédito 22
12. ¿Cómo mejorar el score? 24
13. Supuestos del Modelo 25
Supuestos sobre la Información 25
Supuestos Estadísticos 25
Supuestos Económicos 25
14. Ventajas y Limitaciones del Modelo 26
Ventajas del Modelo 26
Interpretabilidad del Modelo 26
Facilidad de Implementación 26
Limitaciones del Modelo 26
Datos Limpios 26
Rangos de Puntaje 26
Capacidad para Clasificar 27
15. Implementación del Modelo 27
Separación Modular 27
Entrenamiento 27
Output 27
16. Resultados 28
Umbrales de decisión 28
Desempeño del modelo 28
Clase ‘Poor’ 28
Clase ‘Standard’ 29
Clase ‘Good’ 29
Matriz de Confusión 30
Comparación con Benchmark 31
17. Reflexión del Equipo 31
Alejandro Fajardo 31
Diego Lozoya 32

Arantza Gomez Haro 32
18. Conclusiones 33
19. Bibliografía 35
20. Anexos 36
Todas las Gráficas 36
Código 39

> *"El análisis crediticio consiste en transformar información imperfecta en una decisión razonable."*
>
> **- EDWARD ALTMAN -**

1. Introducción

El riesgo crediticio representa una de las principales fuentes de incertidumbre dentro del sistema financiero, ya que está directamente relacionado con la capacidad de los individuos o empresas para cumplir con sus obligaciones de pago. Una estimación inadecuada de este riesgo puede derivar en pérdidas financieras significativas para las instituciones, mientras que una evaluación eficiente contribuye a una mejor asignación de recursos y a una toma de decisiones más informada.

Las instituciones financieras se enfrentan al reto constante de evaluar a solicitantes utilizando información limitada y, en muchos casos, imperfecta. Esta situación obliga a encapsular distintas características individuales en una sola decisión: otorgar o no el crédito. En este contexto, el proceso de evaluación de un crédito se convierte en un proceso crítico, ya que una decisión incorrecta puede implicar fuertes pérdidas financieras y aumentar considerablemente el nivel del riesgo.

Ante este problema, es de suma importancia contar con modelos de crédito tradicionales que permiten evaluar de manera clara, consistente y objetiva. Estos modelos buscan transformar información diversa en una calificación que representa el nivel de riesgo de cada solicitante.

Dichos modelos son utilizados ampliamente debido a su sencillez y facilidad de interpretación. Es importante mencionar que permiten estandarizar el proceso de evaluación crediticia al utilizar criterios objetivos, datos concretos y algoritmos para calificar a los solicitantes, asegurando consistencia en la aprobación y rechazo de créditos.

De esta manera, los modelos de crédito tradicionales se consolidan como una herramienta fundamental dentro del sistema financiero, al proporcionar una base estructurada que apoya la toma de decisiones, gestionando adecuadamente el riesgo crediticio. En este proyecto, se desarrolla un modelo de crédito tradicional con el propósito de evaluar y clasificar a los solicitantes de acuerdo con su nivel de riesgo, así como analizar el desempeño del modelo mediante la comparación de sus resultados con una clasificación previamente establecida.

2. Objetivo del Proyecto

Objetivo General

El objetivo principal de este proyecto es desarrollar un modelo de crédito tradicional que permita evaluar y clasificar el riesgo crediticio de una lista de solicitantes mediante un sistema de

puntaje, con el fin de sustentar la toma de decisiones en la aprobación o rechazo de créditos. Además, se busca analizar el desempeño del modelo comparándolo con la clasificación ya dada en la base de datos utilizada para el análisis, evaluando su capacidad para identificar correctamente el nivel de riesgo de los solicitantes.

Objetivos Específicos

Los objetivos específicos de este proyecto son los siguientes:

- Identificar y seleccionar variables relevantes para la evaluación del riesgo crediticio.
- Definir criterios objetivos y ponderaciones para la elaboración del puntaje crediticio.
- Elaborar un tablero de puntaje de crédito que sintetice la información en una calificación única.
- Implementar un modelo de crédito en Python.
- Generar calificaciones crediticias para cada solicitante a partir del modelo desarrollado.
- Evaluar el desempeño del modelo mediante la comparación con la calificación otorgada en la base de datos.

3. Marco Teórico

Riesgo Crediticio

El riesgo crediticio se define como la posibilidad de que un prestatario no cumpla con sus obligaciones de pago de un contrato financiero, ya sea de manera parcial o total. Dicho esto, el riesgo de crédito representa una de las principales fuentes de incertidumbre para las instituciones financieras, ya que un incumplimiento puede generar pérdidas económicas significativas.

El análisis correcto de este riesgo es fundamental, ya que es una herramienta que permite determinar si se debe de otorgar o no un crédito. Es importante realizar un análisis adecuado del riesgo de crédito ya que:

- Permite la toma de decisiones con mayor seguridad
- Reduce el incumplimiento
- Ayuda a tener una evaluación profunda del perfil del consumidor
- Construye relaciones sólidas con los clientes al ofrecer préstamos adecuados a su situación

Por otro lado, el riesgo crediticio se determina por múltiples factores asociados al comportamiento financiero del solicitante del crédito. Algunos de estos factores incluyen:

- Historial financiero y crediticio
- Aperturas recientes de créditos
- Variables sociodemográficas

- Nivel de endeudamiento
- Actividad financiera

Es importante mencionar que la información disponible para evaluar estos factores suele ser limitada o imperfecta, incrementando la complejidad del proceso de evaluación y aumenta la posibilidad de cometer errores en la toma de decisiones.

Dado este contexto, las instituciones financieras recurren a modelos de evaluación crediticia que permiten la clasificación de los solicitantes según su nivel de riesgo y proporcionan una base objetiva para la aprobación o rechazo de créditos.

Modelos de Crédito Tradicionales

Los modelos de crédito tradicionales son herramientas utilizadas por las instituciones financieras para evaluar y clasificar el riesgo crediticio de los solicitantes a partir de información histórica financiera y características observables. Estos modelos se basan en criterios específicos y en sistemas de puntuación que permiten transformar variables individuales en una clasificación general de riesgo, facilitando la toma de decisiones en la aprobación y rechazo de créditos.

El funcionamiento de estos modelos consiste en asignar una puntuación numérica a cada solicitante con base en un conjunto de variables previamente establecidas. Cada variable utilizada en el modelo tiene un peso distinto la cual al multiplicarse por la variable suma o resta puntos para el score crediticio.

Finalmente, los puntajes obtenidos en cada variable se suman, de manera ponderada, generando una calificación final que representa el nivel de riesgo crediticio del solicitante.

Una de las principales ventajas de los modelos de crédito tradicionales es su simplicidad. En este caso, cada variable que se incluye en el modelo cuenta con una ponderación específica, lo cual permite identificar de manera directa la contribución de cada variable en la evaluación final. Esta interpretabilidad resulta útil ya que permite comprender cómo influyen los distintos factores en la calificación crediticia del solicitante y ayuda en la justificación de las decisiones tomadas.

Además, estos modelos permiten estandarizar el proceso de evaluación crediticia, ya que los mismos criterios y ponderaciones son aplicados a todos los solicitantes. Esto asegura consistencia en la aprobación y rechazo de créditos. De esta manera, se tiene un trato equitativo de todos los solicitantes.

Dicho esto, es importante mencionar que estos modelos también pueden presentar ciertas limitaciones. Su desempeño depende en gran medida de la calidad y disponibilidad de la información de los solicitantes, ya que datos incorrectos o imprecisos pueden afectar la precisión de la calificación obtenida.

FICO Score

Uno de los modelos de crédito tradicionales más reconocidos es el FICO Score, desarrollado por Fair Isaac Corporation. Este modelo es

ampliamente utilizado y sirve como referencia para la evaluación del riesgo crediticio.

El FICO Score genera una puntuación que oscila entre 300 y 850, donde una puntuación más alta indica un menor riesgo crediticio. Esta calificación se construye mediante cinco variables clave, cada una con una ponderación específica que refleja su importancia relativa en la evaluación del riesgo.

Variable	Ponderación
Historial de Pagos	35%
Cantidad Adeudada	30%
Duración del Historial de Crédito	15%
Solicitudes de Crédito Recientes	10%
Combinación de Créditos	10%

Tabla 1: Ponderación del FICO Score

Con base en la puntuación obtenida, considerando las variables anteriores, los solicitantes se clasifican en distintas categorías de riesgo, tal como se muestra a continuación:

Clasificación	Rango de Puntaje
Excepcional	800 - 850
Muy Bueno	740 - 799
Bueno	670 - 739
Razonable	580 - 669
Deficiente	300 - 579

Tabla 2: Tablero de Puntaje de FICO Score

Esta estructura permite a las instituciones financieras tomar decisiones de manera consistente. El éxito de este tipo de modelos radica en su capacidad de condensar información fundamental en un único número interpretable.

En el caso de este proyecto, el FICO Score sirve como referencia conceptual para la construcción del scorecard desarrollado en este proyecto, aunque adaptado a la información disponible en la base de datos analizada.

Tablero de Puntaje de Crédito

El tablero de puntaje de crédito, también conocido como scorecard, es una herramienta clave que permite traducir las características observables de un solicitante, es decir, aquellos atributos medibles y verificables como puede ser su historial de pagos, en una puntuación numérica que refleja su nivel de riesgo crediticio. Como se puede observar en el FICO Score, esta herramienta se compone de dos elementos

principales, los cuales son el rango de puntaje y la clasificación.

Su objetivo principal es sistematizar el proceso de evaluación de manera directa, garantizando, como se mencionó anteriormente, que todos los solicitantes sean evaluados considerando los mismos criterios. Para construir un scorecard, se deben de seleccionar variables relevantes para la predicción del nivel de riesgo crediticio, a las cuales se les asigna un conjunto de rangos de puntaje específico para cada uno. Los puntajes obtenidos con cada variable se suman, generando la calificación general. Esta luego se compara con la tabla de puntaje de crédito para determinar el nivel de riesgo del solicitante y si se le debe otorgar el crédito o no.

Su facilidad de implementación e interpretación lo convierte en una herramienta sumamente utilizada en la industria financiera. Este principio es el que fundamenta el desarrollo del modelo que será presentado en este proyecto, donde se busca transformar las características individuales de cada solicitante en una calificación que refleje adecuadamente su nivel de riesgo de manera objetiva y comparable.

Evaluación del Desempeño de Modelos de Crédito

La evaluación del desempeño de un modelo de crédito es algo esencial para determinar su utilidad y confiabilidad en la toma de decisiones financieras. Un modelo adecuado no solo debe generar una calificación crediticia, sino también demostrar que dicha calificación se encuentra alineada con el comportamiento real o la clasificación ya establecida previamente.

En el caso de este proyecto, la precisión del modelo se obtiene dividiendo el número de clasificaciones correctas entre el total de observaciones analizadas. Los errores del modelo se identifican cuando la categoría predicha por el modelo difiere de la clasificación real del cliente en el conjunto de datos.

Los errores de clasificación pueden dividirse en dos categorías principales:

- Error Tipo I (Falso Positivo): Ocurre cuando el modelo asigna la etiqueta de una clase a un solicitante que no pertenece realmente a ella.
- Error Tipo II (Falso Negativo): Ocurre cuando el modelo no asigna la etiqueta de una clase a un solicitante que sí pertenece a ella.

Dado el carácter multiclase y ordinal del modelo desarrollado con tres segmentos de riesgo diferenciados (‘Poor’, ‘Standard’ y ‘Good’) estos errores no pueden caracterizarse de forma global para todo el modelo, sino que deben interpretarse desde la perspectiva de cada clase individualmente. Un mismo caso de clasificación incorrecta constituye simultáneamente un Error Tipo I para la clase incorrectamente asignada y un Error Tipo II para la clase real del solicitante, dependiendo del segmento desde el cual se analice.

Las implicaciones financieras de estos errores varían según la clase involucrada. Cuando un solicitante perteneciente a la clase ‘*Poor*’ es

clasificado erróneamente en un segmento de '*Good*' (Error Tipo II desde la perspectiva de '*Poor')* el modelo incurre en el error de mayor costo esperado para la institución, al derivar en el otorgamiento de créditos con alta probabilidad de incumplimiento. En sentido contrario, cuando un solicitante de perfil '*Standard*' o '*Good*' es asignado incorrectamente a un segmento *'Poor'* (Error Tipo I desde la perspectiva *'Poor')* el modelo sobreestima el riesgo real, generando el rechazo de créditos potencialmente viables y representando una pérdida de oportunidad de negocio. El análisis de estos errores por clase resulta esencial para evaluar la precisión y consistencia del modelo, así como para validar su aplicabilidad en un entorno real de evaluación crediticia.

4. Descripción del Modelo ADA

Rango del Puntaje

El modelo de crédito desarrollado se encuentra delimitado dentro de un rango de 0 a 600 puntos. Dentro de este sistema de puntuación, los valores se manejan exclusivamente como números enteros, ya que cualquier valor decimal generado durante el cálculo es automáticamente truncado por el modelo.

La interpretación del puntaje es directa: a mayor puntaje, menor es el nivel de riesgo crediticio del solicitante. De esta manera, los clientes que obtienen puntuaciones más altas presentan un perfil financiero más sólido en comparación con aquellos que reciben puntuaciones más bajas.

Clasificación del Riesgo Crediticio

Para facilitar la interpretación del resultado obtenido, el modelo establece dos umbrales de decisión que permiten clasificar a los solicitantes en tres niveles de riesgo crediticio.

La clasificación se realiza de la siguiente manera:

Clasificación	Rango de Puntaje
Poor	**0-220**
Standard	**221 - 410**
Good	**411 - 600**

Tabla 3: Rangos de puntaje para clasificación del Modelo ADA

Esta estructura permite traducir el resultado numérico del modelo en una clasificación cualitativa del riesgo, facilitando la interpretación del perfil crediticio de cada solicitante.

Lógica de Construcción del Puntaje

El puntaje final del modelo se obtiene a partir de la agregación de puntos asociados a distintas características del solicitante. Cada variable considerada en el modelo contribuye de manera específica al resultado final, ya sea incrementado o reduciendo la puntuación total dependiendo de las características observadas en el solicitante.

De esta manera, el modelo transforma información individual de cada variable en un indicador numérico único que representa el nivel de riesgo crediticio del solicitante. Posteriormente, este puntaje es comparado con los umbrales de clasificación establecidos para

determinar la categoría de riesgo correspondiente.

5. Preparación y Limpieza de Datos

El proceso de limpieza de datos implementado corresponde a una estrategia integral de depuración, validación y transformación orientada a garantizar consistencia estructural, coherencia longitudinal por cliente y adecuación para modelado de riesgo crediticio. Las intervenciones realizadas pueden agruparse en cinco grandes bloques:

- Normalización de valores faltantes
- Corrección de formatos
- Validación
- Imputación estructurada
- Transformación de variables

Normalización de Valores Faltantes

Diversos marcadores no estándar, como guiones bajos, cadenas vacías, etiquetas como "NA", "NM" y combinaciones atípicas, fueron convertidos explícitamente a valores nulos (NaN). Asimismo, se eliminaron cadenas corruptas que contenían caracteres especiales o patrones anómalos en variables categóricas sensibles, como identificadores personales y comportamientos de pago. Esta etapa garantiza que los valores ausentes sean tratados de manera uniforme y evita que artefactos del proceso de generación o exportación del dataset se interpreten como categorías válidas.

Corrección de Formatos

Se eliminaron sufijos con guiones bajos, espacios residuales y apóstrofes iniciales que impedían la correcta conversión a tipo numérico. Posteriormente, las variables fueron transformadas explícitamente a formato cuantitativo, forzando a valores no interpretables a convertirse en nulos. Esta etapa reduce el riesgo de errores silenciosos durante el modelado y asegura integridad tipológica.

Validación

Se definieron rangos plausibles, de manera arbitraria, para variables clave como edad, tasa de interés, número de tarjetas, número de préstamos, ingreso anual y utilización de crédito. Los valores que excedían límites razonables fueron anulados. Esta decisión responde a criterios de coherencia económica y evita que outliers espurios distorsionen estimaciones posteriores.

Imputación Estructurada

Se aplicó una estrategia de imputación jerárquica. Para variables numéricas, los valores faltantes se completaron primero mediante la mediana por cliente, preservando así la heterogeneidad individual; en caso de persistir valores nulos, se utilizó la mediana global. Las variables categóricas se imputaron con una categoría explícita denominada "Missing",

permitiendo que la ausencia de información conserve poder predictivo potencial.

Transformación de Variables

La variable de antigüedad crediticia, originalmente en formato textual ("X Years and Y Months"), fue convertida a una medida continua en años. Se implementó interpolación temporal por cliente y un procedimiento de backfilling consistente con incrementos mensuales, garantizando coherencia longitudinal. Variables complejas, como tipo de préstamo y comportamiento de pago, fueron descompuestas en indicadores binarios mediante codificación one-hot, eliminando posteriormente las columnas originales.

Finalmente, se impuso consistencia intra-cliente en variables que no deberían variar mensualmente (ejemplo: número de cuentas o edad), utilizando la moda o el valor más frecuente dentro de cada identificador.

6. Benchmark

Con el objetivo de establecer un punto de referencia que permita evaluar el desempeño del modelo de crédito desarrollado, mejor conocido como benchmark, se implementó una regresión logística simple. Esto funciona como una solución estándar de clasificación y permite capturar de manera objetiva los resultados obtenidos con el modelo propuesto.

Regresión Logística Simple

La regresión logística es un modelo estadístico ampliamente utilizado en situaciones de clasificación binaria, cuyo propósito es estimar la probabilidad de que una observación pertenezca a una de dos categorías posibles.

En este proyecto, la regresión logística se implementa de forma simple, utilizando directamente todas las variables disponibles en el conjunto de datos y sin aplicar técnicas de feature engineering ni ajustes complejos. De esta manera, el modelo funciona exclusivamente como un benchmark, representando un enfoque estándar de clasificación frente al cual se evaluará el desempeño del modelo de crédito tradicional desarrollado.

Resultados del Benchmark

Clasificación	Precisión	Recall	F1-score
Poor	0.51	0.28	0.37
Standard	0.54	0.85	0.66
Good	0.36	0.02	0.04

Tabla 4: Resultados del Benchmark

El análisis por clase revela un desempeño desequilibrado que limita la utilidad práctica del modelo benchmark como instrumento de clasificación crediticia. El accuracy score global de 0.5371 encubre diferencias sustanciales entre segmentos: la clase '*Standard*' concentra la mayor parte de las predicciones correctas con un recall de 0.85, lo que refleja una tendencia del

modelo a sobre clasificar hacia la categoría más frecuente, comportamiento típico de modelos no especializados ante clases desbalanceadas. En contraste, las clases de mayor relevancia crediticia exhiben un desempeño notoriamente inferior: la clase '*Poor*' alcanza un recall de apenas 0.28, implicando que el modelo deja sin identificar al 72% de los clientes con perfil de incumplimiento real, mientras que la clase *Good* registra el valor más bajo del reporte con un recall de 0.02, prácticamente incapaz de distinguir a los solicitantes de bajo riesgo. El F1-score ponderado de 0.47 sintetiza este desempeño global deficiente, particularmente en las clases extremas, cuya correcta identificación constituye el criterio central de cualquier sistema de clasificación crediticia. Estos resultados establecen la línea base frente a la cual se evalúa el modelo desarrollado en el presente trabajo.

7. Selección de Variables

Objetivo de la Selección de Variables

El primero paso en la construcción del modelo de crédito fue la selección de variables relevantes. El objetivo de esta etapa consistió en identificar aquellas variables con mayor influencia sobre la variable objetivo, con el fin de construir un modelo de crédito tradicional basado únicamente en los factores más informativos y consistentes. Una adecuada selección de variables permite mejorar la capacidad explicativa del modelo, reducir el ruido en los datos y facilitar la interpretación de los resultados obtenidos.

Asimismo, esta etapa resulta fundamental para el desarrollo posterior del tablero de puntaje de crédito, ya que permite trabajar con un conjunto reducido de variables que reflejan de manera más precisa el perfil de riesgo de los solicitantes.

Metodología XGBoost

Para apoyar el proceso de selección de variables, se utilizó un algoritmo llamado XGBoost, un modelo supervisado de Machine Learning basado en árboles de decisión. Este algoritmo se empleó exclusivamente con fines exploratorios y como herramienta auxiliar de análisis, y no como el modelo final de evaluación crediticia. Su uso permitió identificar aquellas variables con mayor influencia sobre la variable objetivo, facilitando la construcción posterior de un modelo de crédito tradicional más interpretable.

El funcionamiento de XGBoost se basa en la construcción secuencial de árboles de decisión, donde cada nuevo árbol aprende de los errores cometidos por los árboles anteriores. Este enfoque permite capturar relaciones no lineales y patrones complejos entre las variables explicativas y la variable objetivo, lo cual resulta útil para jerarquizar variables según su relevancia dentro del conjunto de datos. En este análisis, la variable objetivo corresponde a la clasificación crediticia de los solicitantes presente en la base de datos, la cual se utiliza únicamente como referencia para identificar las variables más relevantes.

Este modelo ha resultado ser más preciso que otros modelos de ML según varios reportes y trabajos académicos. NeurIPS documenta que en el trabajo de Chen y Guestrin (2016), el algoritmo XGBoost presenta una mejor capacidad de optimización, reduciendo el riesgo de sobreajuste (overfitting) y permitiendo la construcción de modelos más robustos y con mayor capacidad predictiva.

Interpretación de Variables mediante Valores SHAP

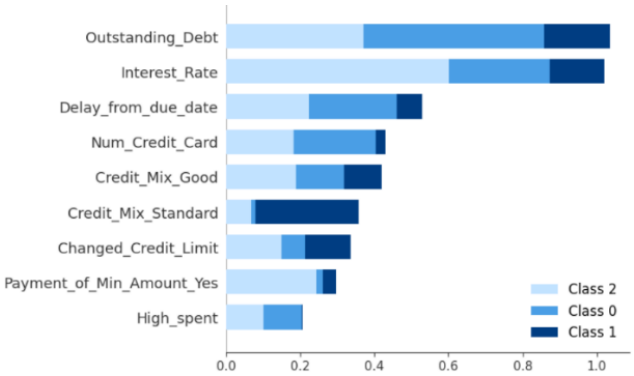
Con el objetivo de interpretar de manera clara y consistente la importancia de las variables identificadas y su impacto en el modelo, se utilizaron valores SHAP. Esta herramienta se fundamenta en principios de teoría de juegos y permite cuantificar la contribución individual de cada variable en las predicciones del modelo.

Los valores SHAP asignan a cada variable un valor que indica su impacto sobre la predicción. Un valor positivo sugiere que la variable contribuye a aumentar la probabilidad de pertenecer a una determinada clase, mientras que un valor negativo indica un efecto contrario. Esta interpretación facilita la comprensión del rol que juega cada variable dentro del proceso de clasificación.

A partir de este análisis, se seleccionaron las variables con mayor impacto y consistencia, las cuales fueron posteriormente utilizadas en la construcción del modelo de crédito tradicional. De esta manera, la selección de variables se realizó de forma objetiva e informada, manteniendo la interpretabilidad y estructura propias de un scorecard crediticio.

Resultados de la Selección de Variables

Tras la aplicación del XGBoost y la implementación de los valores SHAP, se obtuvieron los siguientes resultados:



Gráfica 1: Importancia de las Variables en el Modelo y su División por Clases

La grafica anterior permite observar las nueve variables con mayor importancia dentro del modelo. A partir de esta visualización es posible analizar el impacto que cada variable tiene en la predicción de las distintas clases. Por ejemplo, se puede ver que la variable de Interest Rate tiene un impacto mucho mayor para la clasificación de la clase 2, a diferencia de Outstanding Debt, que su mayor impacto es para la clasificación de la clase 0, o como el caso de Credit Mix Standard, donde su mayor relevancia está en clasificar al solicitante en la clase 1. Esta separación de importancia por clase se hace para cada variable y se suma, y aquellas variables que en conjunto tengan un mayor impacto en el modelo son las seleccionadas.

Este análisis de las características por cada tipo de clase y su importancia reforzó la selección de

las variables más relevantes para la construcción del modelo de crédito.

Las variables seleccionadas fueron las siguientes:

Variables
Outstanding Debt
Interest Rate
Delay from Due Date
Number of Credit Cards
Credit Mix standard
Change Credit Limit
Credit Mix Good
Payment of Min Amount Yes
High Spent

Tabla 5: Variables Seleccionadas

8. Asignación de Signos

Tras analizar la selección de las variables, lo siguiente fue determinar los pesos que tendría cada una de estas en nuestro modelo final. Para realizar esto, primero fue necesario establecer los signos que tendría cada una de las variables a usar con base en los valores de SHAP, sumado a un análisis financiero de cada una.

Outstanding Debt

Descripción de la Variable

La variable Outstanding Debt representa el total de deuda que el solicitante mantiene pendiente de pago. Esta incluye obligaciones financieras como tarjetas de crédito, préstamos personales u otros compromisos financieros. Por lo tanto, esta variable refleja el nivel actual de endeudamiento del solicitante al momento de evaluar su perfil crediticio.

Relevancia para el Riesgo Crediticio

El nivel de deuda pendiente es un indicador importante del riesgo crediticio, ya que permite evaluar la carga financiera con la que ya cuenta el solicitante. Un nivel elevado de deuda de un solicitante puede ser riesgoso, ya que esto implica una capacidad reducida para asumir deudas, aumentando la probabilidad de que enfrente dificultades para cumplir con futuros pagos.

Explicación del Signo

El signo que se le dio a esta variable fue negativo, ya que mientras un cliente tenga una mayor deuda pendiente, su nivel de apalancamiento aumenta. Esto limita su capacidad de contraer deuda nueva, trayendo como consecuencia un incremento es su probabilidad de impago, haciendo que su score crediticio se vea afectado negativamente.

Interest Rate

Descripción de la Variable

La variable Interest Rate hace referencia a la tasa de interés aplicada a los créditos con los que

cuenta el solicitante. Esta tasa representa el costo de financiamiento que el solicitante paga por utilizar recursos prestados por una institución financiera.

Relevancia para el Riesgo Crediticio

La tasa de interés es una variable relevante dentro del análisis del riesgo crediticio, ya que suele reflejar la percepción de riesgo que las instituciones financieras tiene sobre un solicitante. En general, solicitantes con perfiles financieros más sólidos tienden a recibir tasas de interés más bajas, mientras que aquellos con mayor riesgo percibido enfrentan condiciones de financiamiento menos favorables.

Explicación del Signo

El signo aplicado a esta variable fue negativo. La razón de esta decisión es que, normalmente, las tasas de interés altas son asociadas con clientes riesgosos, ya que, como se mencionó previamente, las instituciones financieras tienden a cobrar una tasa más alta a aquellos clientes cuya probabilidad de impago es mayor, con el objetivo de compensar el riesgo asumido. Por lo tanto, a mayor tasa de interés asociada al solicitante, mayor es la probabilidad de riesgo crediticio, lo que impacta negativamente su score dentro del modelo.

Delay from Due Date

Descripción de la Variable

El Delay from Due Date corresponde al número de días que tarda el solicitante en pagar su deuda después de la fecha límite establecida. Este indicador permite evaluar la puntualidad con la que el solicitante cumple con sus obligaciones financieras, proporcionando información sobre su comportamiento de pago.

Relevancia para el Riesgo Crediticio

Un indicador muy relevante en la evaluación del riesgo crediticio son los retrasos en los pagos. Un solicitante que presenta un historial frecuente o prolongado de atrasos refleja una menor disciplina financiera y una menor calidad para cumplir en tiempo y en forma con sus obligaciones. Este tipo de comportamiento incrementa la probabilidad de incumplimiento en el futuro, por lo que resulta un factor clave al momento de evaluar el perfil crediticio de un solicitante.

Explicación del Signo

A esta variable se le asignó un sigo negativo, ya que un mayor número de días de retraso en los pagos está asociado con un mayor nivel de riesgo crediticio. A medida que aumentan los días de atraso, la probabilidad de que el solicitante incumpla con sus obligaciones financieras también incrementa, lo que impacta negativamente su score dentro del modelo.

Number of Credit Cards

Descripción de la Variable

La variable Number of Credit Cards se refiere al número de líneas de crédito que posee el solicitante. Esta variable permite identificar el nivel de acceso que el solicitante tiene ante este tipo de productos financieros.

Relevancia para el Riesgo Crediticio

El número de tarjetas de crédito puede proporcionar información relevante sobre el comportamiento financiero del solicitante. Por un lado, contar con varias tarjetas de crédito puede indicar que el solicitante tiene acceso al sistema financiero y experiencia en el manejo de productos crediticios. Sin embargo, un número elevado de líneas de crédito implica una mayor exposición al endeudamiento y un mayor riesgo de sobreutilización del crédito disponible.

Explicación del Signo

En este modelo, se asignó un signo negativo a esta variable, ya que un mayor número de tarjetas de crédito puede aumentar la probabilidad de endeudamiento de la persona. Al contar con múltiples líneas de crédito disponibles, el solicitante demuestra que necesita mucho dinero prestado, lo cual es algo riesgoso, afectando de manera negativa su score crediticio.

Credit Mix Standard

Descripción de la Variable

La variable Credit Mix Standard señala si el solicitante cuenta con un conjunto estándar de productos crediticios, tales como tarjetas de crédito, préstamos personales o financiamientos similares en su historial financiero. Esto ayuda a identificar tanto la estructura general de las obligaciones financieras que tiene el solicitante, así como la interacción que este ha tenido con el sistema crediticio.

Relevancia para el Riesgo Crediticio

La combinación de distintos productos crediticios que pueda tener el solicitante puede aportar información importante sobre su historial financiero. Contar con una mezcla estándar de tarjetas o préstamos indica que el solicitante ha tenido interacción con estos productos, observando el nivel de experiencia o eficacia que este ha tenido en su historial, viéndose afectado directamente por la cantidad y la capacidad de pago que haya mostrado.

Explicación del Signo

El signo que se asignó a esta variable fue negativo, ya que, contar con una diversificación limitada de productos financieros refleja un historial crediticio menos diversificado. El contar con un historial con poca cantidad de productos financieros dificulta la evaluación crediticia del solicitante. De igual forma, esto genera incertidumbre sobre el comportamiento crediticio que se puede tener, impactando negativamente el modelo de crédito.

Change Credit Limit

Descripción de la Variable

El componente Change Credit Limit hace referencia a las modificaciones que ha hecho la institución financiera al límite de crédito del solicitante. Este tipo de cambios puede significar decisiones tomadas por la entidad con base en el historial de pagos o nivel de endeudamiento que se tiene. Por lo tanto, esta variable permite observar los ajustes a la capacidad de financiamiento otorgada al solicitante.

Relevancia para el Riesgo Crediticio

La variación en el límite de crédito que se le otorga al solicitante es un indicador importante para las instituciones financieras. Un aumento constante en el límite de crédito puede significar más confianza por parte de la institución, derivada de un buen historial de pago. Por otro lado, una disminución en el límite podría significar una revisión o evaluación más cuidadosa y precisa sobre el perfil del solicitante y la disciplina que este tenga al cumplir sus obligaciones financieras.

Explicación del Signo

En el modelo, el signo que se determinó para esta variable fue el positivo. La razón es que, cuando se tiene un mayor incremento en el límite de crédito, generalmente refleja mayor confianza por parte de la institución financiera. Asimismo, este tipo de decisiones suelen basarse en un buen historial de pagos, buena disciplina crediticia y un perfil de riesgo favorable para la institución, lo cual impacta de manera positiva el score crediticio.

Credit Mix Good

Descripción de la Variable

La característica de Credit Mix Good señala si el solicitante tiene una buena combinación de productos financieros dentro de su historial financiero. Esto puede incluir una mezcla de tarjetas de crédito, créditos de consumo, tarjetas personales, etc. Contar con una buena diversidad de productos crediticios bien estructurados puede reflejar una amplia experiencia dentro del sistema crediticio.

Relevancia para el Riesgo Crediticio

Un indicador positivo dentro de los análisis crediticios suele ser el tener una buena combinación de productos financieros. El poseer distintos productos financieros y contar con una gestión integra y sólida de este, no solo puede indicar una amplia experiencia en el sector crediticio, sino también puede estar asociada con un historial crediticio más estable, aumentando la confianza que las instituciones puedan tener sobre el solicitante.

Explicación del Signo

A diferencia del Credit Mix Standard, a esta variable se le puso el signo positivo. Una mezcla diversificada de créditos puede reflejar la amplia experiencia con el manejo de distintas obligaciones financieras. De igual forma, una adecuada gestión de estos tipos de productos financieros puede estar asociada a un historial crediticio estable y sólido, teniendo una consecuencia positiva sobre el score de crédito.

Payment of Min Amount Yes

Descripción de la Variable

La variable de Payment of Min Amount muestra si el solicitante realiza únicamente el pago mínimo de su línea de crédito. Este tipo de pagos indica que, aunque se cumpla con el pago del mínimo, la mayor parte del saldo pendiente permanece sin liquidarse, aumentando la probabilidad de acumulación de intereses. Comportamientos como este reflejan una limitada capacidad de cumplir con todas las obligaciones financieras que el solicitante tiene.

Relevancia para el Riesgo Crediticio

El comportamiento de pago que pueda mostrar el solicitante es un aspecto fundamental para la evaluación crediticia. Cuando el solicitante realiza solo el pago mínimo, parte de la deuda queda pendiente. Este patrón de pago puede indicar que se tiene una limitación para poder cumplir con los pagos correspondientes,

incrementando el riesgo asociado a su perfil crediticio.

Explicación del Signo

Para el pago de la cantidad mínima se asignó el signo negativo. Pagar únicamente el monto mínimo requerido en los créditos puede indicar dificultades para cubrir con toda la deuda pendiente que se tiene. Este tipo de conductas puede representar una mayor presión financiera para el solicitante, impactando negativamente la calificación crediticia que puede llegar a obtener.

High Spent

Descripción de la Variable

El componente High Spent muestra si el solicitante cuenta con un nivel elevado de gasto mediante el uso de su crédito. Un nivel alto de gasto puede significar una mayor utilización de las líneas de crédito a su disposición, marcando una tendencia dentro de su comportamiento financiero.

Relevancia para el Riesgo Crediticio

El nivel de gasto realizado con el uso del crédito otorgado puede ofrecer información valiosa para la toma de decisiones respecto al otorgamiento de crédito. Un gasto elevado puede indicar que el solicitante tiene una participación activa en el sistema financiero, utilizando constantemente sus líneas de crédito, mostrando que tiene experiencia manejando este tipo de productos financieros.

Explicación del Signo

En este modelo se asignó un signo positivo a esta variable, ya que mientras mayor nivel de gasto tenga el solicitante, refleja una mayor actividad financiera del cliente. Además, el contar con una constante actividad en este sistema indica una mayor facilidad para poder acceder a financiamiento, asociándose con un perfil de crédito más sólido, teniendo un impacto positivo sobre el score.

9. Estimación de Pesos

Metodología para la Estimación de Pesos

Una vez seleccionadas las variables relevantes y asegurando que cada una presentara el signo adecuado con respecto al riesgo crediticio, se procedió a entrenar un modelo de regresión logística. A partir de este modelo, se utilizaron los coeficientes asociados a la clase 2 ('Good'), los cuales fueron interpretados como pesos dentro del modelo de puntaje crediticio.

La elección de los coeficientes de esta clase responde al enfoque del modelo, el cual busca identificar a los solicitantes con mejor perfil crediticio. Bajo esta lógica, un mayor puntaje debe estar asociado con una mayor probabilidad de pertenecer a la categoría 'Good'.

Pesos Obtenidos

Los pesos obtenidos para cada variable se presentan en la siguiente tabla:

Variables	Peso
Credit Mix Good	0.725155
Payment of Min Amount Yes	0.385820
Credit Mix Standard	0.349903
High Spent	0.282668
Number of Credit Cards	0.103531
Interest Rate	0.042818
Delay from Due Date	0.019019
Change Credit Limit	0.015326
Outstanding Debt	0.000081

Tabla 6: Variables Seleccionadas y sus Pesos Obtenidos

Como se puede ver en la tabla, cada variable tiene un impacto distinto dentro de la predicción que realiza el modelo. Aquellas que tienen un mayor peso son las variables que impactan más al modelo crediticio propuesto.

Interpretación de las Variables

Credit Mix Good

La variable Credit Mix Good es la característica que tiene más importancia en el modelo. Esto indica que tener una buena diversificación de productos financieros, al igual que una gestión sólida de estos contribuye a un mejor score. Como el enfoque esta direccionado para que los solicitantes que se encuentren dentro de la clase más alta sean los que mejor score tengan, aquellos que tengan un alto valor en Credit Mix Good tendrán más probabilidades de ser clasificados en la categoría "Good", debido a su comportamiento financiero estable y bajo perfil de riesgo.

Payment of Minimum Amount Yes

La segunda variable con más relevancia dentro del modelo es la de Payment of Min Amount Yes. La razón por la cual esta variable tiene un impacto grande es porque indica que el solicitante solo paga el mínimo, indicándole a la institución financiera que se tiene una capacidad limitada para poder cubrir toda la deuda que se tiene, elevando mucho su perfil de riesgo. Al momento de hacer que la variable sea negativa, al multiplicarla con su respectivo peso, esta hace que el score baje significativamente, logrando tener una mejor depuración de clases.

Credit Mix Standard

La tercera variable que más contribuye al modelo es la de Credit Mix Standard. Aunque contar con una combinación normal o estándar de productos crediticios no representa necesariamente un comportamiento negativo, dentro del contexto del modelo su efecto si resulta perjudicial a comparación de tener una buena combinación de productos crediticios. De esta forma, el modelo logra diferenciar bien entre los solicitantes que verdaderamente tienen una buena mezcla de crédito y aspiran a tener una clasificación 'Good' y aquellos que deben tener una clasificación menor.

High Spent

La cuarta variable con más peso es la de High Spent, a la cual se le determinó un peso positivo. Esta importancia sugiere que un mayor nivel de gasto puede estar asociada a una mayor actividad crediticia, dando la interpretación de tener un buen manejo de crédito y constante participación en el sistema financiero. Por esta razón es que tiene un gran impacto para poder tener la mejor clasificación.

Pesos Más Bajos

En cuanto a las otras 5 variables, cada una de ellas presenta un peso relativamente bajo en comparación con las primeras cuatro variables, lo que refleja que, si bien aportan información valiosa para el modelo, el impacto que tienen sobre el score crediticio final es más limitado. Cabe destacar que cada una de ellas tiene un signo negativo, lo que indica que un aumento en estas características reduce el score del solicitante.

10. Puntaje de crédito

Para obtener el puntaje otorgado, se calcula el producto punto de los pesos del modelo y las variables seleccionadas. Este primer número obtenido es un puntaje "crudo", el cual es posteriormente escalado para estar dentro del rango de 0 a 600. Para hacer esto, primero se desplazan todos los puntajes obtenidos:

$$s_i = (\hat{s}_i + |\hat{s}_{min}|) \times 100$$

donde:

- $\hat{s}_i = x_i^T \beta$ es el score crudo.

El término $|\hat{s}_{min}|$ es el mínimo observado en el conjunto de prueba y garantiza que $s_i \geq 0$ para todo i.

Para tener el score definitivo, se realiza la siguiente transformación de todo el conjunto de puntajes calculado:

$$Score_i = \left\lfloor \frac{s_i - \min(s)}{\max(s) - \min(s)} \times 600 \right\rfloor$$

Donde el mínimo y máximo cambiarán dependiendo del conjunto de datos que se desea probar, a diferencia del $|\hat{s}_{min}|$ de la primera transformación que se mantendrá igual para cualquier conjunto de datos usados.

Finalmente, tras observar ya el score es necesario verificar la siguiente restricción:

$$Score_i = \max(0, \min(600, Score_i))$$

Donde se escoge el máximo entre 0 el score final, entendiendo que, si el score llega a ser negativo, automáticamente a ese solicitante se le pondrá un score de 0.

11. Tablero de Puntaje de Crédito

Siguiendo con el contexto del modelo, se decidió utilizar una estructura directa para la construcción del score. En este modelo, se puede interpretar el tablero de puntaje como la multiplicación de los pesos obtenidos por el valor de cada variable. De esta forma, cada una de las variables tiene una aportación directa al score final. Como último paso, se suman cada

uno de los valores obtenidos de las multiplicaciones y se obtiene el score crudo.

Para tener un mejor entendimiento sobre la estructura y funcionamiento del modelo, a continuación, se presenta un ejemplo:

Variables	Peso	Valor cliente	Score
Credit Mix Good	0.725155	1	0.7251
Payment of Min Amount Yes	0.385820	0	0
Credit Mix Standard	0.349903	0	0
High Spent	0.282668	1	0.2826
Number of Credit Cards	0.103531	-5	-0.5176
Interest Rate	0.042818	-7	-0.2997
Delay from Due Date	0.019019	-11	-0.2092
Change Credit Limit	0.015326	11.24	0.1722
Outstanding Debt	0.000081	-1377.74	-0.1115

Tabla 7: Ejemplo de Tablero de Puntaje

Como se observa en la tabla, cada una de las variables por las que está constituida el modelo aparece con su respectivo peso. Este peso es multiplicado por los valores que presenta el cliente en cada una de las características, mismas que ya tienen el signo que se asignó previamente, mostrando los resultados en la columna de "Score". Por último, para obtener el score crudo se suman estos resultados, obteniendo que para el cliente analizado el score crudo fue de 0.0419.

Posteriormente, este score crudo pasa por el proceso de escalamiento, para así obtener ya el score final, mismo que puede ser interpretado y clasificado según los thresholds ya establecidos. Primero se realiza el desplazamiento de datos con la siguiente fórmula:

$$s_i = (\hat{s}_i + |\hat{s}_{min}|) \times 100$$

Sustituyendo:

$$s_i = (0.0419 + 4.26) \times 100$$

$$s_i = 430.19$$

Una vez obtenido este valor, para calcular ya el score que se clasificará, es necesario pasarlo por una transformación con el siguiente procedimiento:

$$Score_i = \left\lfloor \frac{s_i - \min(s)}{\max(s) - \min(s)} \times 600 \right\rfloor$$

Sustituyendo:

$$Score_i = \left\lfloor \frac{430.19 - 0}{538.79 - 0} \times 600 \right\rfloor$$

$$Score_i = 479$$

Antes de clasificar el score resultante de las transformaciones, es importante corroborar que este score cumple con la restricción para la clasificación, la cual es:

$$Score_i = \max(0, \min(600, Score_i))$$

En este caso, como el resultado del score es mayor que 0, este score es el que será utilizado para hacer la clasificación.

Esta clasificación se realiza ubicando el score dentro de los thresholds previamente establecidos. De esta forma es posible saber si el cliente se encuentra en la clase "Poor", "Standard" o "Good".

Tomando en cuenta los thresholds de:

- 0 a 220 es clasificado como "Poor".
- 221 a 410 es clasificado como "Standard"
- 411 a 600 es clasificado como "Good"

Es posible realizar la clasificación y obtener el resultado final:

Gráfica 2: Clasificación de Crédito mediante Puntaje

Como es visible en la imagen, el resultado se encuentra dentro del umbral de 411 a 600, determinando que la clasificación que se le debe dar al solicitante es la de "Good".

12. ¿Cómo mejorar el score?

Como se pudo apreciar en el ejemplo anterior, cada una de las variables tiene un peso importante dentro de la construcción del modelo, y un cambio en cualquiera de las variables puede hacer que el solicitante se encuentre en una clase y no en otra.

En el mundo crediticio, mientras más alto se esté dentro de la clasificación de crédito, mayores beneficios y oportunidades se pueden obtener. Un mejor puntaje puede facilitar el acceso a créditos, tasas de interés competentes y condiciones financieras más favorables. Como el objetivo es mejorar el score para tener este tipo de ventajas, es necesario entender que se puede hacer para tener un score alto.

Basándose en el modelo presentado, para obtener un mejor score es fundamental poner atención a las 4 primeras variables, ya que estas pueden determinar gran parte del puntaje que se puede obtener. Si se desea estar dentro de la clasificación más alta, es recomendable tener una buena combinación de productos crediticios, ya que hará que el score tienda a ser más alto, y, por ende, no se tendrá una combinación de productos crediticios estándar, eliminando directamente una opción para reducir el score.

De igual forma, para obtener el mejor score posible, es conveniente no hacer solamente el pago mínimo y tener un alto gasto o uso de los créditos. De esta forma se evita una de las variables que más afecta negativamente al modelo como lo es el pago mínimo de la deuda, mientras que la variable de gasto alto apoya al aumento del score.

A pesar de que estas variables son las de mayor peso, las otras no se pueden descuidar. Las primeras 4 variables solo pueden recibir de entrada un 1 o 0, por lo cual solo se puede obtener un resultado por variable de 0 o del mismo peso dado a cada una. A diferencia de

esto, las otras 5 pueden recibir cualquier número, haciendo que el rango de resultados sea bastante amplio, pudiendo convertir un score que en las primeras 4 variables era bueno, en uno que lleve a la peor clasificación posible.

Entendiendo esto, para evitar que el score disminuya, se debe procurar que, tanto el número de tarjetas, deuda pendiente y la tasa de interés, sean lo más bajas posibles para evitar que su impacto dentro del modelo sea mayor del que se busca.

En cuanto a los días de retraso después de la fecha límite, esta no solo es importante mantenerla baja, sino que, si se paga antes de la fecha límite, el número crudo sería negativo, y con la asignación del signo negativo, esta se vuelve positiva, pasando a ser una variable que aumente ahora el score. Por otro lado, se tiene que cuidar que la variable de cambio en la línea de crédito siempre sea positiva para que aumente el score, ya que, así como sucede con los días de retraso, si se tiene un cambio en el crédito negativo, la variable ahora restará al score, haciendo que este sea menor de lo buscado.

Si se busca obtener el mejor puntaje crediticio y con ello la mejor clasificación, no basta solo con tener las variables que suman en buen nivel, sino tener la mejor combinación de cada una de ellas. Se debe buscar que las variables que restan estén lo más bajas posibles, y las que aumentan el score mantenerlas en un nivel óptimo, de esta forma se logrará el objetivo de estar en la mejor clasificación de crédito.

13. Supuestos del Modelo

Supuestos sobre la Información

El modelo asume que la información utilizada para la evaluación de los solicitantes es completa, consistente y correctamente registrada. Asimismo, se supone que las variables utilizadas durante el entrenamiento del modelo estarán disponibles para nuevas observaciones y serán calculadas de la misma forma que en el conjunto de datos original. En caso de que alguna variable no esté disponible o presente ciertos errores, el cálculo del puntaje crediticio podrá verse afectado.

Supuestos Estadísticos

Desde un punto de vista más estadístico, se asume que las observaciones utilizadas para entrenar el modelo son representativas de la población de solicitantes de crédito. Asimismo, se considera que las relaciones entre variables y la probabilidad de pertenecer a cada categoría de riesgo pueden ser capturados adecuadamente, mediante el modelo de regresión logística utilizado para estimar los pesos. Además, se supone que las observaciones, es decir, el análisis de cada solicitante es evaluada de manera independiente dentro del modelo.

Supuestos Económicos

El modelo parte del supuesto de que las variables seleccionadas para la evaluación reflejan comportamientos financieros relevantes. Por ejemplo, se asume que características como el

endeudamiento y el comportamiento de pago del solicitante influyen en su probabilidad de incumplimiento.

Implicaciones de los Supuestos

El cumplimiento de los supuestos mencionados anteriormente es fundamental para garantizar la validez de los resultados del modelo. Si la calidad de los datos no es la adecuada, las variables no se miden de forma consistente o el modelo se aplica a una población con características significativamente diferentes, el desempeño del modelo podría verse afectado gravemente y sus predicciones podrían perder precisión.

14. Ventajas y Limitaciones del Modelo

Ventajas del Modelo

Interpretabilidad del Modelo

Al tratarse de un modelo de crédito tradicional basado en un sistema de puntaje, una de sus principales ventajas es su alto nivel de interpretabilidad. Cada variable incluida dentro del modelo posee un peso definido dentro de la puntuación total, lo que permite identificar con claridad que factores tienen mayor influencia en la evaluación del solicitante.

Además, cada variable se encuentra dividida en categorías específicas que suman o restan puntos al puntaje final, lo que permite analizar de manera directa como ciertas características del cliente afectan positiva o negativamente su evaluación crediticia. Esto facilita comprender las razones detrás de la calificación obtenida por cada solicitante.

Facilidad de Implementación

Otra ventaja importante del modelo es su simplicidad operativa. Al utilizar un rango de puntuación acotado, números enteros y umbrales explícitos de decisión, el modelo resulta fácil de comunicar, interpretar e integrar dentro de sistemas de evaluación crediticia.

Asimismo, este tipo de modelo puede aplicarse de manera masiva a grandes volúmenes de solicitudes, manteniendo un bajo costo marginal de evaluación. Esto lo convierte en una herramienta practica y eficiente para instituciones financieras que requieren evaluar múltiples solicitudes de crédito de manera consistente.

Limitaciones del Modelo

Datos Limpios

Una de las principales limitaciones es que el modelo requiere datos limpios y estructurados, utilizando las mismas variables con las que fue construido. En caso de que algunas de las variables necesarias no estén disponibles para un solicitante, el modelo no podrá calcular su puntaje final correctamente.

Rangos de Puntaje

Otra limitación importante es que la clasificación se basa en rangos discretos de puntaje, lo que puede generar situaciones donde clientes con perfiles de riesgo diferentes terminen clasificados dentro de la misma

categoría. Por ejemplo, un cliente con un puntaje de 221 y otro con 410 serían clasificados como 'Standard', a pesar de que su nivel de riesgo podría ser considerablemente distinto.

Capacidad para Clasificar

Durante el análisis de resultados se observó cierto solapamiento entre el puntaje obtenido y la clasificación real de los clientes, lo que indica que el modelo no logra separar perfectamente a las distintas categorías de riesgo. Esto sugiere que, aunque el modelo captura parte de la información relevante para la predicción, aún existen limitaciones en su capacidad para clasificar con total precisión el riesgo crediticio de los solicitantes, mostrando una gran área de mejora.

15. Implementación del Modelo

Separación Modular

El modelo de crédito fue implementando íntegramente en Python separado en cinco módulos con responsabilidades delimitadas. Estos módulos son los siguientes:

- **Fitter:** Gestiona el entrenamiento y persistencia de los modelos (benchmark y modelo de crédito).
- **Utils:** Ejecuta la selección y negación de variables.
- **Credit Model:** Concentra la lógica de scoring. Usando el modelo de fitter, convierte las variables de utils en el puntaje de 0 a 600.
- **Visualizations:** Genera los reportes gráficos de evaluación.
- **Main:** Actúa como orquestador del pipeline completo.

Esta separación modular permite que el entrenamiento del modelo, del cual se obtienen los pesos, ocurra una única vez, mientras que la inferencia sobre nuevos clientes opera de forma independiente y automatizada.

Entrenamiento

El entrenamiento se realiza mediante una regresión logística ajustada sobre el conjunto de entrenamiento. Los coeficientes de esta regresión son guardados con ayuda de joblib. Fitter funciona como pieza clave en el sistema, ya que al clasificar nuevos clientes (como los del conjunto de prueba), el modelo recupera automáticamente los pesos guardados con joblib, sin necesidad de reentrenar, de modo que se garantiza consistencia en el modelo a lo largo del tiempo.

Output

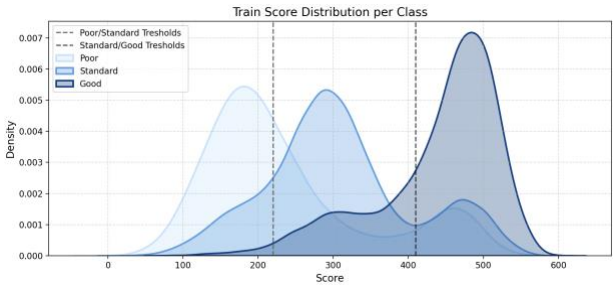
El output del sistema no es únicamente una etiqueta de clase, sino un score continuo que captura la gradualidad del riesgo crediticio. La clasificación final se deriva de dos umbrales definidos sobre la distribución del score: clientes con score ≤ 220 son clasificados como 'Poor', entre 220 y 410 como 'Standard' y con score ≥ 410 como 'Good'. Esta representación continua añade valor operacional al permitir ajustar los

criterios de clasificación sin necesidad de reentrenar el modelo, así como identificar casos en zonas limítrofes que podrían requerir revisión adicional.

16. Resultados

Umbrales de decisión

Para elegir los umbrales de decisión del modelo y poder dividir entre clases, se graficó el score obtenido por clase real, como se muestra en la siguiente imagen:



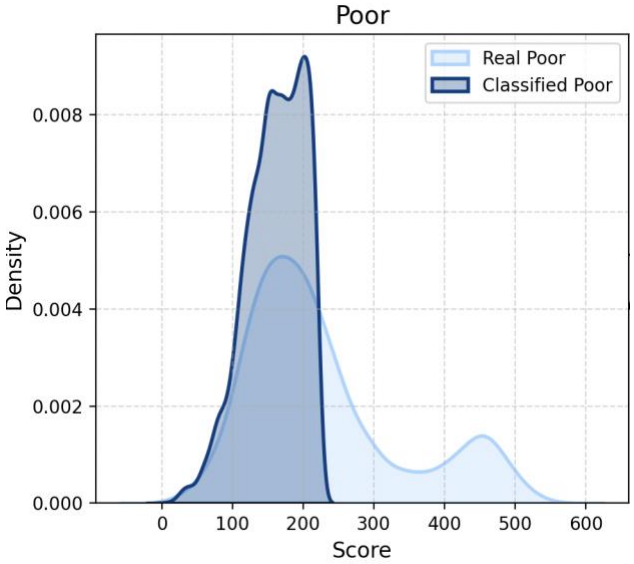
Gráfica 3: Score Obtenido por Clase Real

Las tres clases se solapan y no hay un límite en el que se pueda diferenciar claramente el cambio de clases, no obstante, se puede ver una clara concentración por clase en puntos en específico. La definición de los umbrales de decisión fue con base en una aproximación visual, cerca de la intersección de las distribuciones entre clases. Estos umbrales finalmente fueron colocados en los valores de score 220 (para diferenciar entre 'Poor' y 'Standard') y 410 (para diferenciar entre 'Standard' y 'Good').

Desempeño del modelo

Con base en los umbrales definidos, la distribución de la clasificación del modelo de crédito, en comparación con la clasificación real para el conjunto de prueba es la siguiente:

Clase 'Poor'

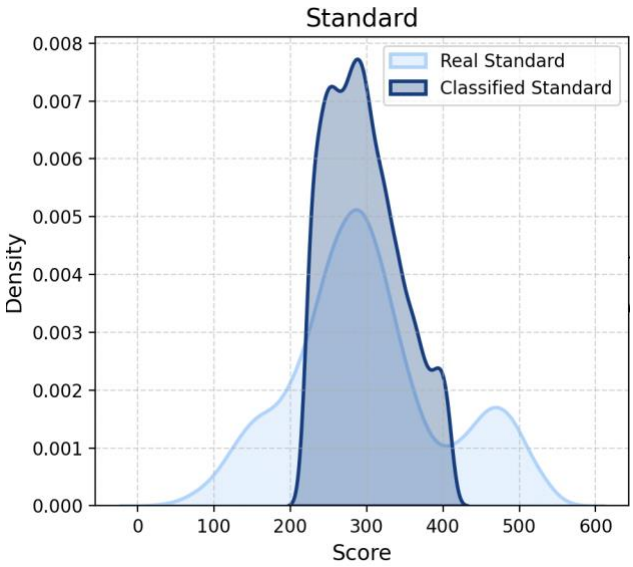


Gráfica 4: Distribución de la Clasificación de la Clase 'Poor'

La distribución de puntuaciones correspondiente a la clase 'Poor' revela un patrón de clasificación con concentración marcada en el segmento inferior del rango de scoring. La distribución de clientes realmente clasificados como Poor ('Classified Poor') presenta una concentración pronunciada en torno a los 200 puntos, con una asimetría positiva que se extiende hacia valores intermedios del rango. En contraste, la distribución de los clientes verdaderamente pertenecientes a la clase Poor ('Real Poor') exhibe mayor dispersión, con una cola derecha que alcanza valores superiores a los 400 puntos. Esta parte de la cola derecha de la distribución 'Real Poor', que se extiende más allá del umbral de clasificación (220 puntos), representa la principal fuente de error Tipo II: clientes con perfil de incumplimiento real cuya

puntuación los ubica fuera del segmento Poor, siendo clasificados erróneamente en categorías de menor riesgo.

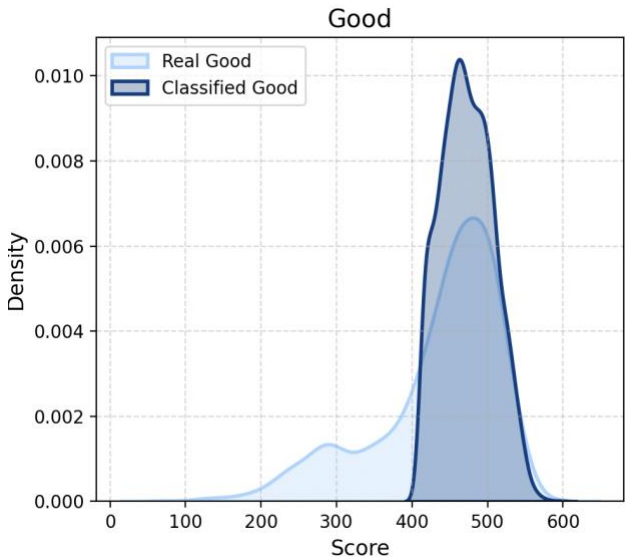
Clase 'Standard'



Gráfica 5: Distribución de la Clasificación de la Clase 'Standard'

La distribución de puntuaciones correspondiente a la clase 'Standard' presenta una dinámica clasificatoria más compleja, derivada de su posición intermedia entre los dos umbrales de decisión (220 y 410 puntos). La distribución 'Classified Standard' exhibe una concentración pronunciada en torno a los 300 puntos, con una forma aproximadamente simétrica y colas delgadas que se contienen dentro del intervalo definido. En contraste, la distribución 'Real Standard' muestra mayor dispersión en ambas direcciones, con colas que se extienden tanto por debajo de los 220 puntos como por encima de los 410. La cola izquierda que cae por debajo del umbral inferior y la cola derecha que supera el umbral superior constituyen conjuntamente la fuente de error Tipo II del modelo para esta clase: clientes con perfil real Standard cuya puntuación los desplaza fuera del intervalo de clasificación, siendo asignados erróneamente a los segmentos 'Poor' o 'Good' respectivamente.

Clase 'Good'



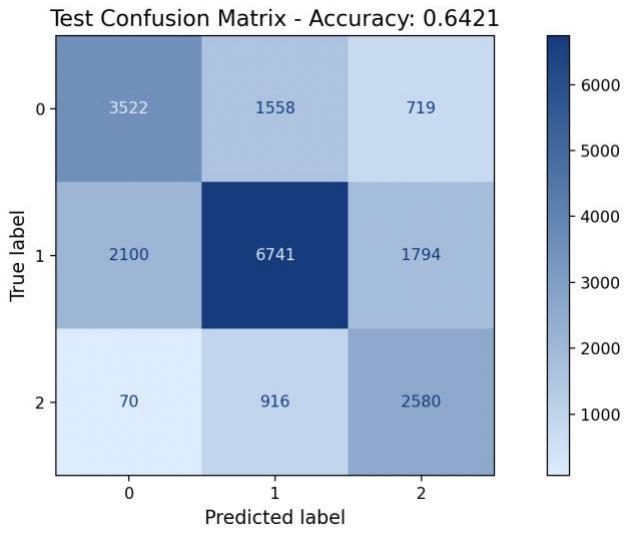
Gráfica 6: Distribución de la Clasificación de la Clase 'Good'

La distribución de puntuaciones correspondiente a la clase 'Good' refleja un comportamiento simétrico al observado en la clase 'Poor', en tanto que el modelo opera sobre un único umbral de decisión (410 puntos) que delimita el segmento superior del rango de puntuación. La distribución 'Classified Good' presenta una concentración pronunciada en torno a los 450 puntos, con una asimetría negativa moderada y una cola izquierda que se contiene en el umbral. En contraste, la distribución 'Real Good' exhibe mayor dispersión, con una cola izquierda que se extiende por debajo de los 410 puntos, constituyendo la principal fuente de error Tipo II para esta clase: clientes con perfil real de bajo riesgo cuya puntuación los ubica por debajo del

umbral superior, siendo clasificados erróneamente en el segmento 'Standard' o inclusive 'Poor'.

Matriz de Confusión

Para poder evaluar el desempeño del modelo, es necesario tomar en consideración las tres clases de manera simultánea, lo cual se ve representado en la matriz de confusión y reporte de clasificación del conjunto de prueba:



Gráfica 7: Matriz de Confusión

Clasificación	Precisión	Recall	F1-score
Poor	0.62	0.61	0.61
Standard	0.73	0.63	0.68
Good	0.51	0.72	0.60

Tabla 8: Reporte de Clasificación del Conjunto de Prueba

La evaluación del modelo sobre el conjunto de prueba arroja un accuracy score global de 64.21%, resultado consistente con el nivel de complejidad inherente a modelos de scorecard clásicos aplicados sobre variables de comportamiento crediticio discretizadas. El reporte de clasificación revela un desempeño diferenciado por clase:

- 'Standard' presenta la mayor precisión (0.73), lo que indica que cuando el modelo asigna un cliente a este segmento lo hace con relativa confiabilidad; sin embargo, su recall (0.63) refleja la pérdida bilateral documentada en su distribución de puntuaciones, con 2,100 casos desplazados hacia 'Poor' y 1,794 hacia 'Good'.
- La clase 'Good' exhibe el recall más alto (0.72) pero la precisión más baja (0.51), sugiriendo que el modelo tiende a sobre clasificar en este segmento, es decir, asigna puntuaciones superiores a 410 a clientes que no necesariamente pertenecen a la clase de menor riesgo.
- La clase 'Poor' representa la limitación más relevante desde una perspectiva de gestión crediticia: con un recall de 0.61, aproximadamente 4 de cada 10 clientes con perfil de incumplimiento real no son capturados por el modelo, siendo desplazados principalmente hacia el segmento 'Standard' (1,558 casos) y en menor medida hacia 'Good' (719 casos).

Estos errores Tipo II implican la exposición de la cartera a riesgo no identificado, cuyo costo esperado supera típicamente al de los errores de sobre clasificación. El F1-score ponderado de 0.65 sintetiza un desempeño global moderado,

consistente con los estándares documentados para modelos de scorecard clásicos en contextos de clasificación crediticia, donde la capacidad discriminatoria entre perfiles de riesgo constituye el criterio de evaluación central.

Comparación con Benchmark

La comparación entre el modelo desarrollado y el modelo benchmark evidencia una mejora sustancial en todas las métricas de desempeño. La exactitud global aumentó de 0.54 a 0.64, representando una ganancia de 10 puntos porcentuales, mientras que el F1-score ponderado se incrementó de 0.47 a 0.65, reflejando una clasificación más equilibrada entre las tres clases.

Las mejoras más significativas se concentran precisamente en las clases de mayor relevancia crediticia. La clase 'Poor' registra el avance más notable en términos de captura de riesgo real: el recall se incrementó de 0.28 a 0.61, más que duplicando la capacidad del modelo para identificar clientes con perfil de incumplimiento, mientras que la precisión pasó de 0.51 a 0.62. La clase 'Good' exhibe la mejora más dramática del reporte: su recall aumentó de 0.02 a 0.72, corrigiendo la incapacidad casi total del modelo de referencia para distinguir a los solicitantes de bajo riesgo. La clase 'Standard', que en el benchmark concentraba artificialmente la mayor parte de las predicciones correctas, mantiene un desempeño estable con una mejora en precisión de 0.54 a 0.73, al mismo tiempo que su recall se ajusta de 0.85 a 0.63, reducción esperada y deseable, indicativa de que el modelo propuesto distribuye las clasificaciones de forma más balanceada entre los tres segmentos en lugar de sobre clasificar hacia la categoría mayoritaria.

En conjunto, los resultados demuestran que el modelo de crédito clásico desarrollado supera consistentemente al modelo de referencia en todas las dimensiones de evaluación relevantes para la toma de decisiones crediticias.

17. Reflexión del Equipo

Alejandro Fajardo

El principal de los retos en lo personal fue la creación del modelo. Si bien, toda la elaboración del documento, así como la justificación del cómo o por qué se hizo de cierta forma fue complicado, creo que el reto verdadero fue encontrar un modelo que pudiera superar a nuestro benchmark y tuviera una justificación lógica.

Otro reto importante fue comprender a profundidad la construcción de los modelos clásicos. Conforme el trabajo iba desarrollándose, se volvió más claro que no solo es importante tener un buen accuracy en tu modelo, sino que este debe ser interpretable en todo aspecto. En el sistema crediticio, entender como cada una de las variables afectan a los modelos y con ello la toma de decisiones es fundamental, ya que no solo justifica las clasificaciones de crédito que se otorgan, sino también funcionan como guía para las personas, donde pueden entender en que rubros no están cumpliendo con lo requerido para estar en una

mejor clase, así como saber en cuales de ellos están en un nivel aceptable para mantenerlo en este mismo.

A pesar de que si hubo presencia de limitantes, considero que fue un proyecto que ayudó a entender la importancia de los modelos clásicos en el mundo crediticio, y el por qué, a pesar de ser modelos muy simples, estos siguen siendo los más usados en el mundo, donde la capacidad de interpretación y facilidad para justificar las decisiones tomadas son igual de importantes como el mismo desempeño predictivo.

Diego Lozoya

En lo personal, subestimé la complejidad del proyecto. Desde el comienzo asumí que simplemente íbamos a leer el dataset y crear un modelo de manera bastante rápida, sin embargo, al juntarnos a trabajar por primera vez nos dimos cuenta de que el dataset proporcionado no estaba limpio (factor al que no estamos acostumbrados). Nos llevó todo un día de trabajo en equipo para poder tener el dataset limpio y ahora si comenzar a trabajar en el modelo de crédito.

Otro de los grandes retos fue la creación como tal del modelo. Una vez habiendo seleccionado las variables más útiles para el modelo, no sabíamos cómo construir el mismo, por lo que buscando en internet encontramos distintas ideas de cómo hacerlo. Uno de los problemas de esa fase es que el modelo que estábamos haciendo era demasiado complejo y usaba herramientas que ni nosotros sabíamos explicar al 100%, alejándonos del objetivo de construcción de un modelo de crédito clásico.

Tras reflexionar sobre esto y eliminar los factores más complejos, no solo mejoro la interpretación del modelo, sino que también mejoró el desempeño del mismo. Con este nuevo modelo nos sentíamos mucho más confiados pues es muy fácil de explicar e interpretar tanto para un usuario promedio (sabiendo que aspectos debe cambiar para mejorar) como para una compañía que busca implementarlo (entendiendo fácilmente la construcción técnica del score y su clasificación).

En general, creo que fue un proyecto muy ilustrativo, pues nos ayudó a comprender en su totalidad, el funcionamiento de modelos de score crediticios usados actualmente. Al crear nuestro propio modelo, aprendimos por prueba y error, algunos de los problemas y su solución en la creación de este tipo de modelos, así como la clasificación de riesgo de los clientes.

Arantza Gomez Haro

El desarrollo de este proyecto representó una oportunidad para aplicar de manera práctica diversos conceptos relacionados con la evaluación del riesgo crediticio. A lo largo del trabajo, uno de los principales retos fue comprender el proceso completo detrás de la construcción de un modelo crediticio, ya que inicialmente parecía una tarea más directa de lo que realmente resultó ser. Con el avance del proyecto, se volvió evidente que no solo se trata de aplicar técnicas que ya hemos aprendido en la carrera, sino también fue fundamental hacer una

limpieza de datos, justificar nuestras decisiones metodológicas y realmente entender la lógica detrás de cada variable seleccionada para el modelo.

Otro reto que se presentó fue la interpretación de los resultados y la evaluación del desempeño del modelo. En este caso, fue muy importante entender que métricas como la precisión no son el único criterio relevante, sino que también es esencial analizar los tipos de errores que pueden generarse y las implicaciones que estos tienen para la toma de decisiones.

Con este proyecto, logré realmente comprender cómo se construyen y evalúan los modelos de crédito tradicionales utilizados en la práctica. Además, el proyecto me hizo realmente valorar la importancia de la calidad y limpieza de datos, ya que esa parte del trabajo fue algo tardado.

En general, considero que este proyecto fue una experiencia muy enriquecedora, ya que me permitió utilizar conocimientos que he obtenido a través de la carrera y que no sabía bien como aplicarlos en situaciones reales. También, considero que este proyecto fue de gran ayuda para entender bien cómo funcionan los modelos crediticios actuales y comprender a fondo como debemos mejorar nosotros nuestros puntajes.

18. Conclusiones

El presente proyecto logró desarrollar e implementar exitosamente un modelo de crédito tradicional capaz de evaluar y clasificar el riesgo crediticio de solicitantes mediante un sistema de puntaje escalado de 0 a 600 puntos. El objetivo general fue cumplido al construir un scorecard interpretable, sustentado en variables seleccionadas con criterio financiero y estadístico, cuyo output permite tomar decisiones de aprobación o rechazo de crédito de manera objetiva y reproducible.

En términos de desempeño, el modelo ADA alcanzó un accuracy global de 64.21% en el conjunto de prueba y un F1-score ponderado de 0.65, representando una mejora sustancial frente al benchmark de regresión logística simple, que operaba con un accuracy de 53.71% y un F1-score de 0.47. Las ganancias más significativas se concentraron en las clases de mayor relevancia crediticia: el recall de la clase 'Poor' más que se duplicó al pasar de 0.28 a 0.61, mientras que la clase 'Good' corrigió prácticamente su incapacidad total de detección, elevando su recall de 0.02 a 0.72. Estos resultados confirman que la combinación de selección de variables mediante XGBoost y SHAP, la asignación de signos con fundamento financiero, y la estimación de pesos a través de los coeficientes de regresión logística constituye un enfoque metodológicamente sólido para la construcción de modelos de scorecard clásicos.

No obstante, el modelo presenta limitaciones relevantes que deben considerarse para su aplicación práctica. La principal de ellas es la capacidad de separación imperfecta entre clases: las distribuciones de puntaje se solapan considerablemente, lo que impide una clasificación sin errores independientemente de los umbrales utilizados. Adicionalmente, el modelo depende de datos limpios y

estructurados, siendo sensible a la calidad de la información de entrada. El uso de rangos de clasificación discretos también genera casos frontera donde solicitantes con perfiles de riesgo distintos reciben la misma etiqueta.

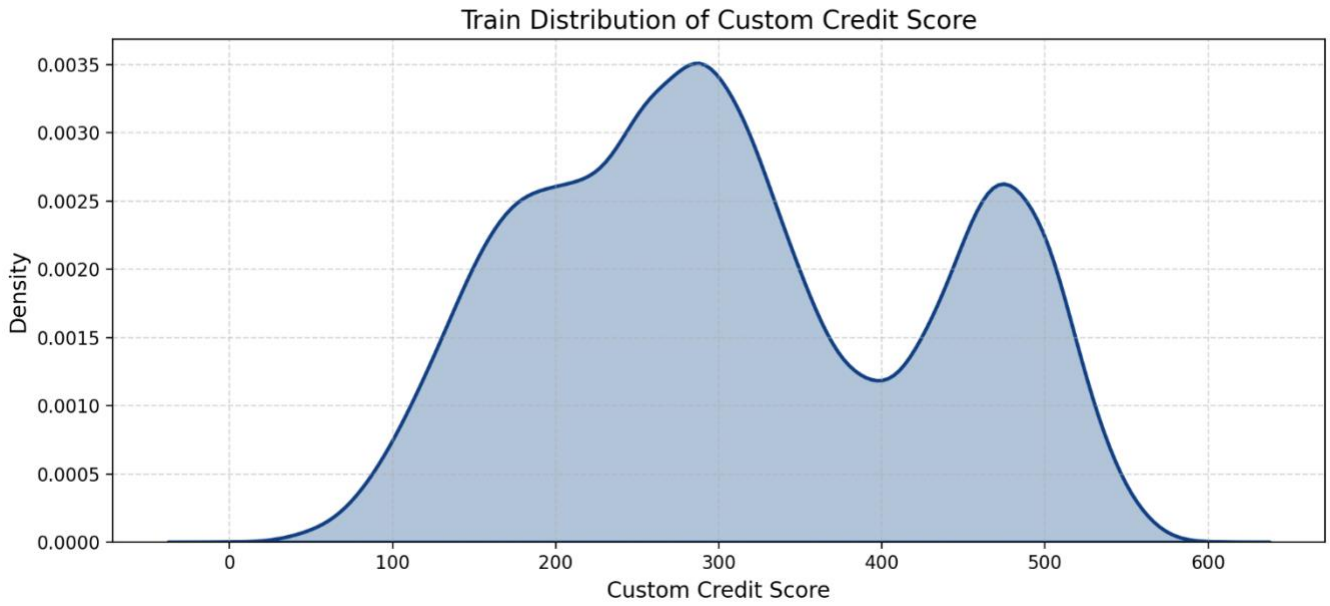
En síntesis, el modelo desarrollado demuestra que los enfoques de crédito tradicionales, cuando son construidos con rigor metodológico y fundamento económico, son capaces de superar soluciones de referencia estándar y ofrecer clasificaciones interpretables y accionables. Su valor no reside únicamente en su capacidad predictiva, sino en la transparencia de sus criterios de evaluación, que permiten comunicar con claridad las razones detrás de cada decisión crediticia.

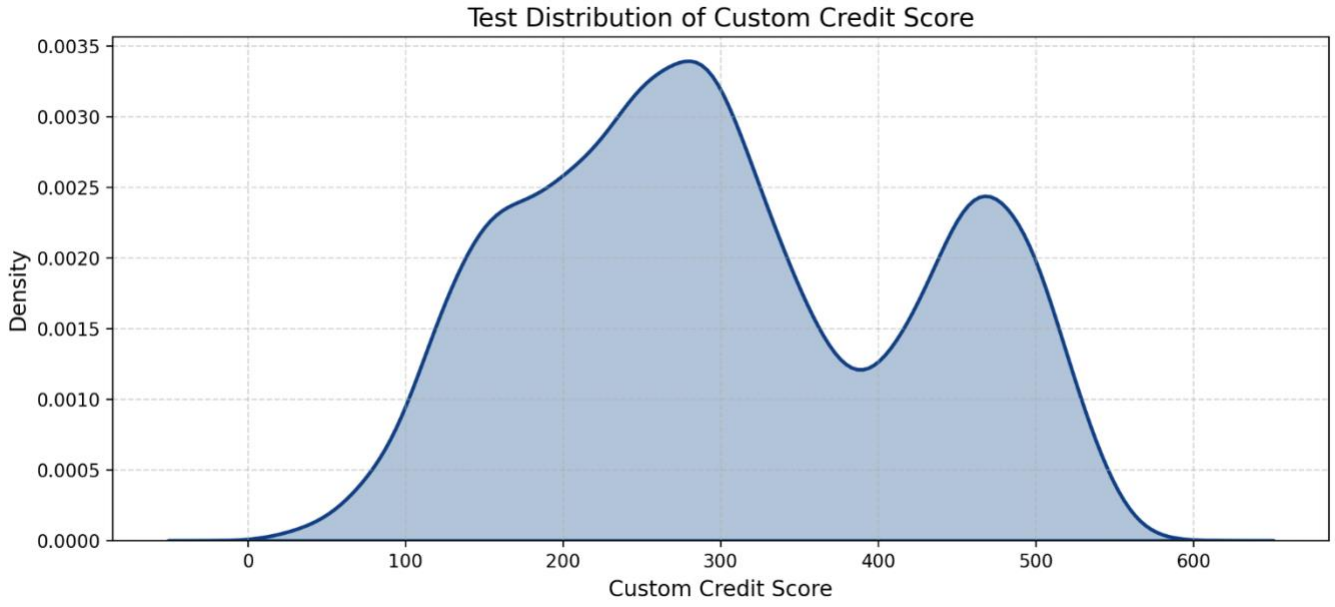
19. Bibliografía

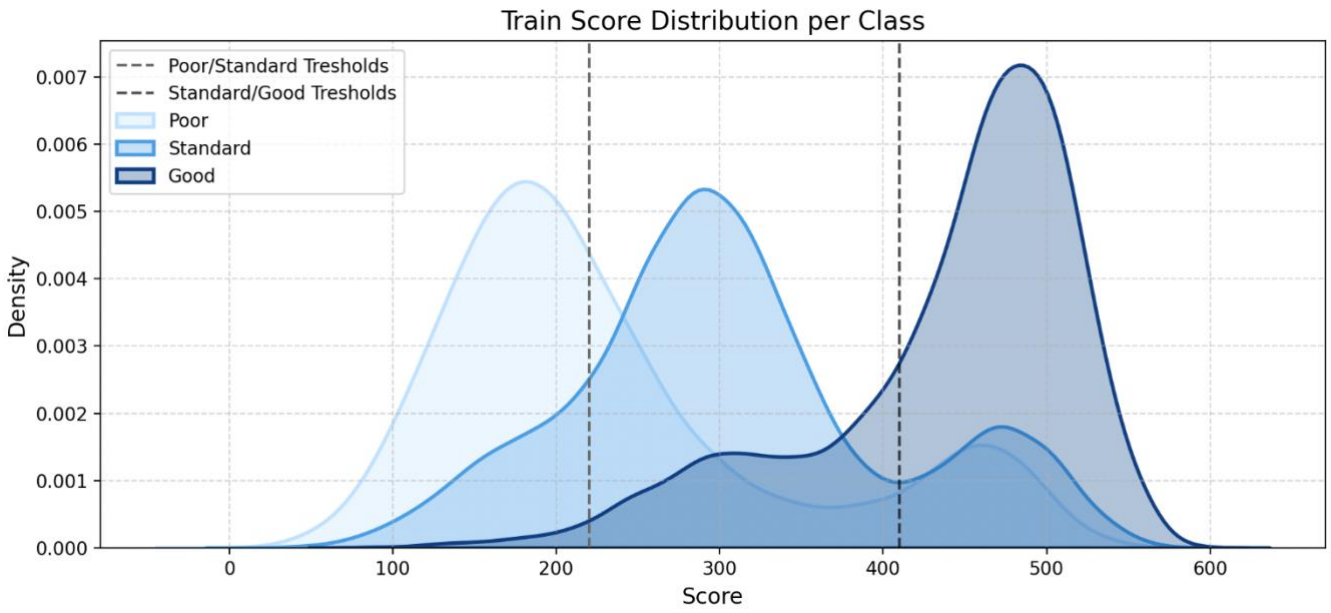
- Candian, M. (2024, September 11). *Riesgo de Crédito: Qué Es, cómo funciona y cómo evaluarlo*. SYDLE. https://www.sydle.com/es/blog/riesgo-de-credito-66c3576d3064907f04634f08
- Experian, D. (2026, February 9). *Riesgo crediticio: Qué Es, por qué importa y cómo puede afectar a tu empresa*. Datablog. https://www.datacredito.com.co/blogs/datablog/riesgo-crediticio-que-es-por-que-importa-y-como-puede-afectar-a-tu-empresa/
- *Credit scoring models: Definition, technologies, and approach done*. Credit Scoring Models: Definition, Technologies, and Approach done. (2026, March 5). https://riskseal.io/glossary/credit-model?utm_
- Luthi, B. (2025, April 4). *FICO® scores: ¿Qué son y en qué se diferencian de Otros Puntajes de Crédito?*. Intuit Credit Karma. https://www.creditkarma.com/es/tarjetas-de-credito/i/puntaje-de-credito-fico
- *¿Qué es un puntaje de fico?: Oficina para la Protección Financiera del Consumidor*. Consumer Financial Protection Bureau. (n.d.). https://www.consumerfinance.gov/es/obtener-respuestas/que-es-un-puntaje-de-fico-es-1883/
- Kavlakoglu, E., & Russi, E. (2025, November 26). *¿Qué es XGBoost?*. IBM. https://www.ibm.com/mx-es/think/topics/xgboost?utm_source=chatgpt.com
- What is logistic regression? - logistic regression model explained - AWS. (n.d.-f). https://aws.amazon.com/what-is/logistic-regression/
- An introduction to shap values and machine learning interpretability | datacamp. (n.d.-b). https://www.datacamp.com/tutorial/introduction-to-shap-values-machine-learning-interpretability
- Credit risk scorecards: Developing and implementing ... (n.d.-c). https://support.sas.com/content/dam/SAS/support/en/books/credit-risk-scorecards/59376_excerpt.pdf

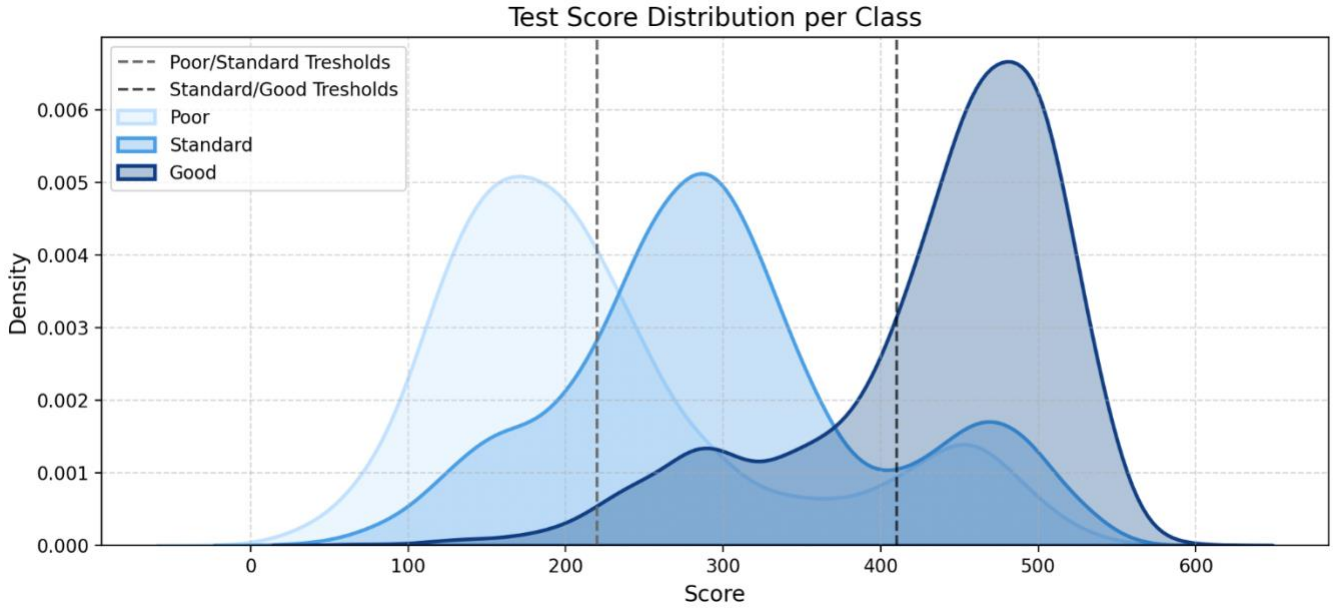
20. Anexos

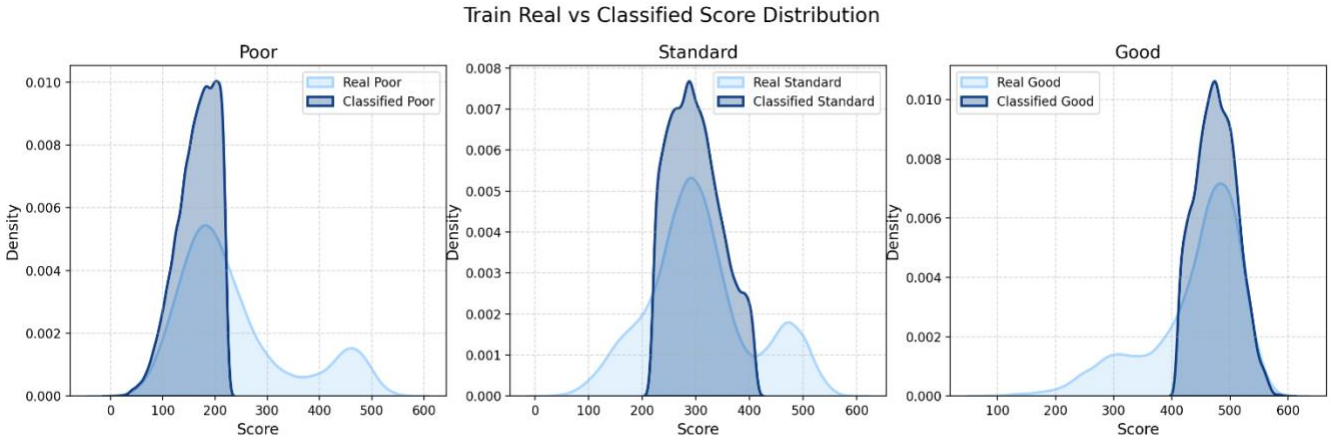
Todas las Gráficas

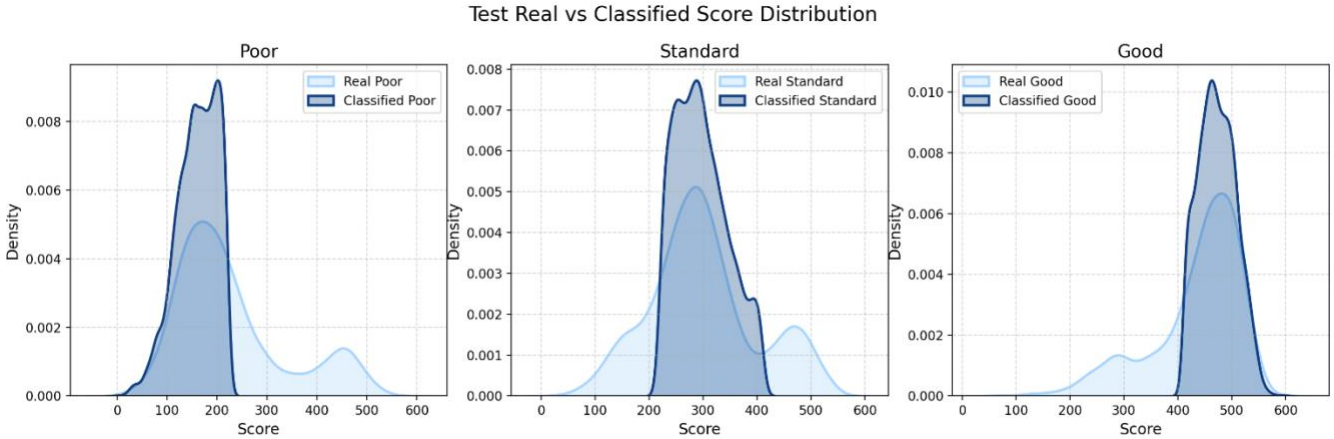


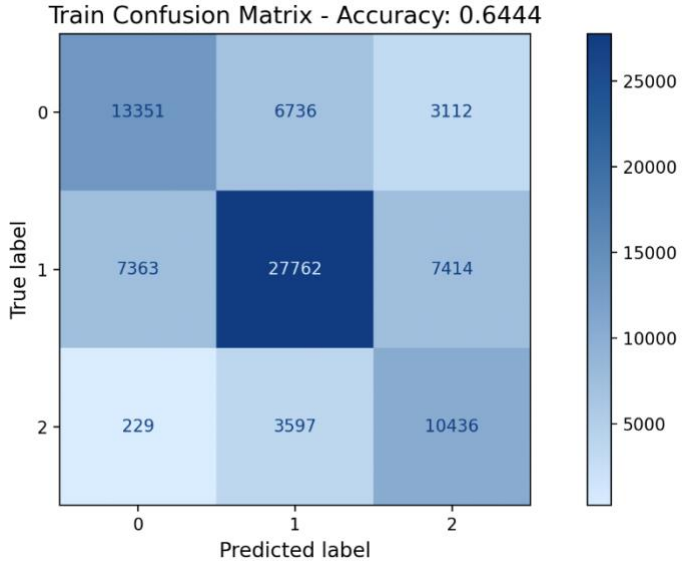


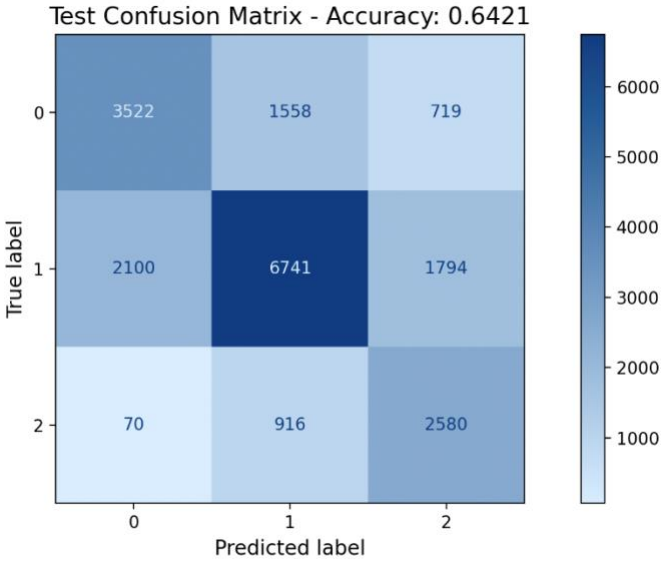

Train Score Distribution per Class
Poor/Standard Tresholds
Standard/Good Tresholds
Poor
Standard
Good
Density
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
0
100
200
300
400
500
600
Score


Test Score Distribution per Class
Poor/Standard Tresholds
Standard/Good Tresholds
Poor
Standard
Good
Density
0.006
0.005
0.004
0.003
0.002
0.001
0.000
0
100
200
300
400
500
600
Score


Train Real vs Classified Score Distribution
Poor
Real Poor
Classified Poor
Standard
Real Standard
Classified Standard
Good
Real Good
Classified Good
Density
Score


Test Real vs Classified Score Distribution
Poor
Real Poor
Classified Poor
Standard
Real Standard
Classified Standard
Good
Real Good
Classified Good
Density
Score


Train Confusion Matrix - Accuracy: 0.6444
13351
6736
3112
7363
27762
7414
229
3597
10436
True label
Predicted label


Test Confusion Matrix - Accuracy: 0.6421
3522
1558
719
2100
6741
1794
70
916
2580
True label
Predicted label

Código

Complete modular codes, datasets, trained models & features selection notebook on 'CreditScoringModel' GitHub Repository: https://github.com/DiegoLozoya1511/CreditScoringModel

data_cleaner.py

```
"""
Credit Dataset Cleaner
======================
Detects and fixes common data quality issues in the
credit scoring dataset.
Generates a cleaned CSV.
"""

import pandas as pd
import numpy as np
import re
from copy import deepcopy

# ── 1. LOAD ──────────────────────────────────────────────────────────

print("Loading dataset...")
df = pd.read_csv("data/train.csv",
low_memory=False)
original_df = df.copy()
report = []        # will collect human-readable log
entries
change_counts = {}  # column → number of cells
changed

def log(msg):
    print(msg)
    report.append(msg)

log(f"\n{'='*60}")
log(f"  CREDIT DATA CLEANING REPORT")
log(f"{'='*60}")
log(f"Rows: {len(df):,}  |  Columns:
{len(df.columns)}")

# ── 2. HELPERS ───────────────────────────────────────────────────────

def strip_trailing_junk(series):
    """Remove trailing underscores / non-numeric
suffixes from numeric strings."""
    return series.astype(str).str.replace(r'[_\s]+$', '',
regex=True)

def is_corrupted_string(val,
pattern=r'^[#@!%\$\^&\*]+|__'):
    """True if the string looks like garbage (special-
char noise or __ prefix)."""
    return bool(re.search(pattern, str(val)))

def count_changes(col_before, col_after):
    return (col_before.astype(str) !=
col_after.astype(str)).sum()

# ── 3. GENERIC PLACEHOLDER → NaN ─────────────────────────────────────

log("\n[1] Converting placeholder markers to NaN")
placeholders = ['_', 'NA', 'NM', '_______', '__10000__',
'nan', 'NaN', '']
```

```
for col in df.columns:
    before = df[col].copy()
    df[col] = df[col].replace(placeholders, np.nan)
    # Also catch strings that are only underscores
    mask = df[col].astype(str).str.fullmatch(r'_+')
    df.loc[mask, col] = np.nan
    changed = count_changes(before, df[col])
    if changed:
        log(f"  {col}: {changed} placeholder(s) → NaN")
        change_counts[col] = change_counts.get(col, 0)
+ changed


# ── 4. CORRUPTED STRINGS → NaN
──────────────────────────────────────────────────────────────────────────────
log("\n[2] Removing corrupted / garbage strings")
string_cols = ['SSN', 'Payment_Behaviour',
'Occupation', 'Credit_Mix']
corrupt_pattern = r'[#@!%\$\^&\*]|^__'

for col in string_cols:
    if col not in df.columns:
        continue
    before = df[col].copy()
    mask =
df[col].astype(str).str.contains(corrupt_pattern,
regex=True, na=False)
    df.loc[mask, col] = np.nan
    changed = mask.sum()
    if changed:
        log(f"  {col}: {changed} corrupted value(s) →
NaN")
        change_counts[col] = change_counts.get(col, 0)
+ changed


# ── 5. STRIP TRAILING UNDERSCORES FROM
NUMERIC COLUMNS
──────────────────────────────────────────────────────────────────────────────
log("\n[3] Stripping trailing underscores from numeric
fields")
numeric_candidates = [
    'Age', 'Annual_Income', 'Monthly_Inhand_Salary',
'Num_Bank_Accounts',
    'Num_Credit_Card', 'Interest_Rate',
'Num_of_Loan', 'Delay_from_due_date',
    'Num_of_Delayed_Payment',
'Changed_Credit_Limit', 'Num_Credit_Inquiries',
    'Outstanding_Debt', 'Credit_Utilization_Ratio',
'Total_EMI_per_month',
    'Amount_invested_monthly', 'Monthly_Balance'
]

for col in numeric_candidates:
    if col not in df.columns:
        continue
    before = df[col].copy()
    cleaned = df[col].astype(str).str.replace(r'_+$', '',
regex=True)
    df[col] = pd.to_numeric(cleaned, errors='coerce')
    changed = count_changes(before, df[col])
    if changed:
        log(f"  {col}: {changed} value(s)
stripped/coerced")
        change_counts[col] = change_counts.get(col, 0)
+ changed
```

```
# Make the rest of the numeric cols proper numeric too
for col in numeric_candidates:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

# ── 6. DOMAIN-SPECIFIC VALIDATION ──────────────────────────────────────────────────────────────
log("\n[4] Domain validation – clamping / nullifying out-of-range values")

def nullify_out_of_range(col, low=None, high=None, label=""):
    if col not in df.columns:
        return
    before = df[col].copy()
    mask = pd.Series(False, index=df.index)
    if low is not None:
        mask |= df[col] < low
    if high is not None:
        mask |= df[col] > high
    df.loc[mask, col] = np.nan
    changed = mask.sum()
    if changed:
        log(f"  {col}: {changed} value(s) out of range {label} → NaN")
        change_counts[col] = change_counts.get(col, 0) + changed

# Age: must be 18–100
nullify_out_of_range('Age', low=0, high=100, label="[18, 100]")

# Interest Rate: must be > 0 and realistic (≤ 60%)
nullify_out_of_range('Interest_Rate', low=0.00, high=60, label="(0, 60]")

# Num_Credit_Card: realistic max ~20
nullify_out_of_range('Num_Credit_Card', low=0, high=20, label="[0, 20]")

# Num_of_Loan: 0–20
nullify_out_of_range('Num_of_Loan', low=0, high=20, label="[0, 20]")

# Delay_from_due_date: can be negative (paid early) but not extreme
nullify_out_of_range('Delay_from_due_date', low=-30, high=365, label="[-30, 365]")

# Num_of_Delayed_Payment: 0–100
nullify_out_of_range('Num_of_Delayed_Payment', low=0, high=100, label="[0, 100]")

# Annual_Income: > 0 and < 10M
nullify_out_of_range('Annual_Income', low=1, high=10_000_000, label="[1, 10M]")

# Monthly_Inhand_Salary: > 0 and < 1M
nullify_out_of_range('Monthly_Inhand_Salary', low=1, high=1_000_000, label="[1, 1M]")

# Amount_invested_monthly: ≥ 0
nullify_out_of_range('Amount_invested_monthly', low=0, label="[0, ∞)")

# Credit_Utilization_Ratio: 0–100 %
```

```
nullify_out_of_range('Credit_Utilization_Ratio', low=0, high=100, label="[0, 100]")

# ── 7. PAYMENT_OF_MIN_AMOUNT → YES/NO ──────────────────────────────────────────
log("\n[5] Standardising Payment_of_Min_Amount (NM → NaN)")
if 'Payment_of_Min_Amount' in df.columns:
    before = df['Payment_of_Min_Amount'].copy()
    df['Payment_of_Min_Amount'] = df['Payment_of_Min_Amount'].replace({'NM': np.nan})
    valid_values = {'Yes', 'No'}
    mask = ~df['Payment_of_Min_Amount'].isin(valid_values) & df['Payment_of_Min_Amount'].notna()
    df.loc[mask, 'Payment_of_Min_Amount'] = np.nan
    changed = count_changes(before, df['Payment_of_Min_Amount'])
    if changed:
        log(f"  Payment_of_Min_Amount: {changed} invalid value(s) → NaN")
        change_counts['Payment_of_Min_Amount'] = change_counts.get('Payment_of_Min_Amount', 0) + changed

# ── 8. IMPUTATION (median for numeric, "Missing" for categorical) ────────────────
log("\n[6] Imputing NaN values with per-customer median (numeric) and 'Missing' (categorical)")
# ── Numeric Imputation ──────────────────────────────────────────────────────────
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
numeric_cols = [col for col in numeric_cols if col != 'Age']  # explicitly exclude Age

for col in numeric_cols:
    null_count = df[col].isna().sum()
    if null_count == 0:
        continue

    # Per-customer median
    if 'Customer_ID' in df.columns:
        df[col] = df.groupby('Customer_ID')[col].transform(
            lambda x: x.fillna(x.median())
        )

    # Global median fallback
    remaining = df[col].isna().sum()
    if remaining > 0:
        df[col] = df[col].fillna(df[col].median())

    filled = null_count - df[col].isna().sum()
    if filled > 0:
        log(f"  {col}: {filled} NaN(s) filled via median")

# ── Categorical Imputation ──────────────────────────────────────────────────────
cat_cols = ['Credit_Mix', 'Payment_Behaviour', 'Payment_of_Min_Amount']
```

```
for col in cat_cols:
    if col not in df.columns:
        continue

    null_count = df[col].isna().sum()
    if null_count == 0:
        continue

    df[col] = df[col].fillna("Missing")

    filled = null_count - df[col].isna().sum()
    if filled > 0:
        log(f"   {col}: {filled} NaN(s) filled with 'Missing'")


# ── FURTHER CHANGES ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
    except:
        return np.nan

df['Credit_History_Age'] =
df['Credit_History_Age'].apply(parse_credit_history_s
imple)

# Use to fill the NaN values on the column
Credit_History_Age by backfilling and subtracting 1
month for each month back.
def bfill_subtract_monthly(group):
    """
    For leading NaNs, backfill by subtracting 1/12
from the first valid value
    """
    # Find first non-null value
    first_valid_idx = group.first_valid_index()

    if first_valid_idx is None:
        return group  # All NaN, can't fill

    # Get position of first valid value
    first_value = group.loc[first_valid_idx]
    first_position = group.index.get_loc(first_valid_idx)

    # Fill all positions before first valid value
    for i in range(first_position - 1, -1, -1):
        idx = group.index[i]
        if pd.isna(group.loc[idx]):
            months_back = first_position - i
            group.loc[idx] = first_value - (months_back /
12)

    return group
```

```
# Filling NaN values in a forward way by customer
by adding 1 month.
df['Credit_History_Age'] =
df.groupby('Customer_ID')['Credit_History_Age'].tran
sform(
    lambda x: x.interpolate(method='linear')
)

# Filling NaN values in a backward way by customer
by subtracting 1 month.
df['Credit_History_Age'] = df.groupby('Customer_ID',
group_keys=False)['Credit_History_Age'].apply(
    bfill_subtract_monthly
)

# --- FILL BLANK SPACES ---
# Step 1: fill from same Customer_ID
df['Type_of_Loan'] =
df.groupby('Customer_ID')['Type_of_Loan'].transform
(lambda x: x.ffill().bfill())

# Step 2: remaining blanks → "Not Specified"
df['Type_of_Loan'] = df['Type_of_Loan'].fillna('Not
Specified')

# --- PERSISTENT ERRORS ---
cols = ['Num_Bank_Accounts', 'Num_Credit_Card',
'Interest_Rate', 'Num_of_Loan']

for c in cols:
    mode_per_customer = (
        df.groupby('Customer_ID')[c]
            .agg(lambda x: x.dropna().mode().iloc[0] if not
x.dropna().mode().empty else np.nan)
    )
    df[c] =
df['Customer_ID'].map(mode_per_customer)


def edad_corregida(grupo):
    conteo = grupo['Age'].value_counts()
    # si hay empate, toma la edad mayor
    edad_final = conteo[conteo ==
conteo.max()].index.max()
    grupo['Age'] = edad_final
    return grupo

df = df.groupby('Customer_ID',
group_keys=False).apply(edad_corregida)

# --- STRAY APOSTROPHES DELETION ---
def delete_stray_apostrophes(column):
    return (
        pd.to_numeric(
            column.astype(str)
                .str.strip()
                .str.lstrip("'"),   # removes leading
apostrophes only
            errors="coerce"
        )
    )

cols = [
    'Monthly_Inhand_Salary',
    'Credit_Utilization_Ratio',
    'Total_EMI_per_month',
```

```
    'Amount_invested_monthly',
    'Monthly_Balance',
    'Credit_History_Age'
]

df[cols] = df[cols].apply(delete_stray_apostrophes)

# --- loan types & payment behaviour handling
(onehot encoding) ---
# - Loan Types -
loan_strings = df['Type_of_Loan'].dropna()

all_loan_types = []

for loan_string in loan_strings:
    loan_string = loan_string.replace(' and ', ', ')
    loans = loan_string.split(',')
    loans = [loan.strip() for loan in loans if loan.strip()]
    all_loan_types.extend(loans)

unique_loan_types = sorted(set(all_loan_types))

for loan_type in unique_loan_types:
    col_name = f'{loan_type.replace(" ", "_").replace("-
", "_")}'

    # Check if loan type exists in Type_of_Loan string
    df[col_name] = df['Type_of_Loan'].apply(
        lambda x: True if pd.notna(x) and loan_type in x
else False
    )

# Drop the original Type_of_Loan column as it's now
encoded into multiple boolean columns
df.drop(columns=['Type_of_Loan'], inplace=True)

# - Payment Behaviour -
behaviour_strings =
df['Payment_Behaviour'].dropna()

all_spend_types = []
all_value_types = []

for behaviour in behaviour_strings:
    if behaviour == "Missing":
        continue

    parts = behaviour.split('_')

    # Expected format:
    # ['High', 'spent', 'Medium', 'value', 'payments']

    if len(parts) >= 4:
        spend_type = parts[0] + "_spent"
        value_type = parts[2] + "_value"

        all_spend_types.append(spend_type)
        all_value_types.append(value_type)

unique_spend = sorted(set(all_spend_types))
unique_value = sorted(set(all_value_types))

for spend in unique_spend:
    col_name = spend

    df[col_name] = df['Payment_Behaviour'].apply(
        lambda x: True if pd.notna(x) and spend in x
else False
```

```
    )

for value in unique_value:
    col_name = value

    df[col_name] = df['Payment_Behaviour'].apply(
        lambda x: True if pd.notna(x) and value in x
else False
    )

df['Behaviour_Missing'] = df['Payment_Behaviour']
== "Missing"

# Drop the original Payment_Behaviour column as
it's now encoded into multiple boolean columns
df.drop(columns=['Payment_Behaviour'],
inplace=True)

# ── 9. DUPLICATE ROWS
──────────────
```

feature_selection.py (originaly .ipynb notebook)

```
# %% [markdown]
# # Features Exploration & Selection
#
# ## Libraries

# %%
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import numpy as np
import pandas as pd
import shap
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier

import warnings
warnings.filterwarnings('ignore')

plt.rcParams['figure.figsize'] = (12, 5)
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.5
plt.rcParams['grid.linestyle'] = '--'
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 12

blue_colors = [
    "#D9EDFF",
    "#C1E2FF",
    "#A8D5FF",
    "#8AC6FF",
    "#6BB6FF",
    "#4A9EE5",
    "#2E86DE",
    "#0066C0",
    "#0055A5",
    "#003D82"
]

blue_scale =
mcolors.LinearSegmentedColormap.from_list(
    "blue_scale",
    [blue_colors[0], blue_colors[-1]]
)

# %% [markdown]
# ## Functions

# %%
def histogram(probabilities: np.array, y_test:
np.array) -> None:
    """
    Plot the probability density of the predicted
probabilities for the correct class.

    Parameters:
        probabilities: numpy array of shape
(n_samples, n_classes) containing predicted
probabilities.
        y_test: numpy array of shape (n_samples,)
containing true class labels.
    """
    plt.figure()
    color_idx = [1, 5, 9]
    for class_idx in range(3):
        sns.kdeplot(
```

```
        probabilities[y_test == class_idx, class_idx],
        color=blue_colors[color_idx[class_idx]],
        fill=True,
        alpha=0.3,
        linewidth=2,
        label=f"Class {class_idx}"
    )

    plt.title("Probability Density (Correct Class)")
    plt.xlabel("Predicted Probability")
    plt.ylabel("Density")
    plt.legend()
    plt.show()


def importance(features: list, importances: list, n_var:
int, importance_type: str) -> None:
    """
    Plot the feature importance based on the provided
importances.

    Parameters:
        features: list of feature names.
        importances: list or array of feature importance
values.
        n_var: number of top variables to display.
        importance_type: string indicating the type of
importance (e.g., 'gain', 'weight').
    """
    model_importance = pd.DataFrame({
        'Variable': features,
        'Importance': importances
    })
    model_importance =
model_importance.sort_values(
        by='Importance', ascending=False).head(n_var)

    cmap =
mcolors.LinearSegmentedColormap.from_list(
        "custom_coolwarm", [blue_colors[-1],
blue_colors[0]])
    custom_palette = sns.color_palette(
        [cmap(i) for i in np.linspace(0, 1,
len(model_importance))])

    plt.figure()
    sns.barplot(x="Importance", y="Variable",
                data=model_importance,
palette=custom_palette)
    for index, value in
enumerate(model_importance["Importance"]):
        plt.text(value - 0.001, index, f"{value:.2f}",
va="center", ha='right')
    plt.xlabel('Importance')
    plt.xlim(0, max(model_importance["Importance"]) *
1.05)
    plt.ylabel('Variables')
    plt.title(f'{n_var} most important variables
({importance_type}-type)')
    plt.show()


def shap_plot(X_test: pd.DataFrame, model:
XGBClassifier, n_var: int) -> np.array:

    X_shap = X_test.sample(int(len(X_test) * 0.8),
random_state=42)
```

```
    explainer = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(X_shap)

    # --- Definir mapeo explícito de colores por clase ---
    color_idx = [1, 5, 9]  # mismos índices que usaste antes

    # ===== GLOBAL MULTICLASS BAR PLOT =====
    plt.figure()
    shap.summary_plot(
        shap_values,
        X_shap,
        plot_type="bar",
        max_display=n_var,
        show=False,
        color=lambda i: blue_colors[color_idx[i]]
    )
    plt.title("SHAP Summary Bar Plot - All Classes")
    plt.show()

    # ===== PLOTS POR CLASE =====
    n_classes = shap_values.shape[2]

    for class_idx in range(n_classes):

        # --- Bar plot individual ---
        plt.figure()
        shap.summary_plot(
            shap_values[:, :, class_idx],
            X_shap,
            plot_type="bar",
            max_display=n_var,
            show=False,
            color=blue_colors[color_idx[class_idx]]
        )
        plt.title(f"SHAP Summary Bar Plot - Class {class_idx}")
        plt.show()

        # --- Dot plot individual ---
        plt.figure()
        shap.summary_plot(
            shap_values[:, :, class_idx],
            X_shap,
            plot_type="dot",
            max_display=n_var,
            cmap=blue_scale,
            show=False
        )
        plt.title(f"SHAP Summary Dot Plot - Class {class_idx}")
        plt.show()

    return shap_values

# %% [markdown]
# ## Classification Model

# %%
def xgboost(X_train: pd.DataFrame, X_test: pd.DataFrame, y_train: pd.Series, y_test: pd.Series) -> tuple[XGBClassifier, np.array]:
    """
    Train an XGBoost classifier and return the model along with predicted probabilities for the test set.
```

```
    Parameters:
        X_train: pd.DataFrame containing the training features.
        X_test: pd.DataFrame containing the test features.
        y_train: pd.Series containing the training labels.
        y_test: pd.Series containing the test labels.

    Returns:
        model: trained XGBClassifier model.
        probabilities: numpy array of shape (n_samples, n_classes) containing predicted probabilities for the test set.
    """
    model = XGBClassifier(
        n_estimators=400,
        max_depth=3,
        learning_rate=0.05,
        reg_lambda=1,
        subsample=0.8,
        colsample_bytree=0.8,
        gamma=0,
        tree_method='hist',
        random_state=42
    )
    model.fit(X_train, y_train, eval_set=[
            (X_train, y_train), (X_test, y_test)], verbose=100)

    probabilities = model.predict_proba(X_test)
    return model, probabilities

# %% [markdown]
# ## Results

# %%
df = pd.read_csv('Data/clean_train.csv')

df['Credit_Score'] = df['Credit_Score'].map({
    'Poor': 0,
    'Standard': 1,
    'Good': 2
})

target = 'Credit_Score'
y = df[target]

# Important features for class diferentation
important_features = ['Outstanding_Debt', 'Interest_Rate', 'Delay_from_due_date', 'Num_Credit_Card',
                      'Credit_Mix_Standard', 'Changed_Credit_Limit', 'Credit_Mix_Good',
                      'Payment_of_Min_Amount_Yes', 'High_spent']

# droping non-informative and target columns
X = df.drop(columns=["Customer_ID", "ID", "Name", "SSN", "Month", target])
# keeping only important features for class differentiation
X = X.drop(columns=[col for col in X.columns if col not in important_features])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
model, probabilities = xgboost(X_train, X_test,
y_train, y_test)

# %% [markdown]
# ### Probabilities histogram

# %%
histogram(probabilities, y_test)

# %% [markdown]
# ### Importances

# %%
importancia_weight_dict =
model.get_booster().get_score(importance_type='we
ight')
importancia_gain_dict =
model.get_booster().get_score(importance_type='gai
n')

# Separar nombres y valores
features_weight =
list(importancia_weight_dict.keys())
valores_weight =
list(importancia_weight_dict.values())

features_gain = list(importancia_gain_dict.keys())
valores_gain = list(importancia_gain_dict.values())

importance(features_weight, valores_weight,
len(important_features), 'weight')
importance(features_gain, valores_gain,
len(important_features), 'gain')

# %% [markdown]
# ### Shap Values

# %%
shap_values = shap_plot(X_test, model,
len(important_features))

# %%
# Negate features for coherent score direction
(higher score = better customer).
negative_features = ['Interest_Rate',
'Outstanding_Debt', 'Delay_from_due_date',
                'Payment_of_Min_Amount_Yes',
'Credit_Mix_Standard',
                'Num_Credit_Card']
```

utils.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,
classification_report


def get_benchmark(model, df: pd.DataFrame) ->
tuple[float, float, str]:
    """
    Evaluates the benchmark model on the provided
DataFrame.

    Parameters:
        model: The pre-trained benchmark model to
evaluate.
```

```
        df (pd.DataFrame): The DataFrame containing the data for evaluation.

    Returns:
        tuple: A tuple containing the accuracy and classification report of the model.
    """
    df = df.copy()

    df['Credit_Score'] = df['Credit_Score'].map({
        'Poor': 0,
        'Standard': 1,
        'Good': 2
    })

    target = 'Credit_Score'
    y = df[target]

    X = df.drop(columns=["Customer_ID", "ID", "Name", "SSN", "Month", target])

    _, X_test, _, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    class_report = classification_report(y_test, y_pred, target_names=[
                                         'Poor', 'Standard', 'Good'])

    return acc, class_report
```

```
def data_splitter(df: pd.DataFrame, target: str) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]:
    """
    Selects and negates features for model, splits the DataFrame into training and testing sets.

    Parameters:
        df (pd.DataFrame): The input DataFrame containing the data for training and testing.
        target (str): The name of the target column in the DataFrame.

    Returns:
        tuple: A tuple containing the training features, testing features, training labels, and testing labels.
    """
    df = df.copy()

    df[target] = df[target].map({
        'Poor': 0,
        'Standard': 1,
        'Good': 2
    })

    target = 'Credit_Score'
    y = df[target]

    # Important features (exploration on features_selection.ipynb)
    important_features = ['Outstanding_Debt', 'Interest_Rate', 'Delay_from_due_date', 'Num_Credit_Card',
```

```
                    'Credit_Mix_Standard',
'Changed_Credit_Limit', 'Credit_Mix_Good',
                    'Payment_of_Min_Amount_Yes',
'High_spent']

    # droping non-informative and target columns
    X = df.drop(columns=["Customer_ID", "ID",
"Name", "SSN", "Month", target])
    # keeping only important features for class
differentiation
    X = X.drop(
        columns=[col for col in X.columns if col not in
important_features])

    # Negate features for coherent score direction
(higher score = better customer).
    negative_features = ['Interest_Rate',
'Outstanding_Debt', 'Delay_from_due_date',
                    'Payment_of_Min_Amount_Yes',
'Credit_Mix_Standard',
                    'Num_Credit_Card']
    X[negative_features] = X[negative_features] * -1

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    return X_train, X_test, y_train, y_test
```

fitter.py

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

from utils import data_splitter


def benchmark_fitter():
    """
    Fit a simple logistic regression model on the raw
features to establish a benchmark.
    """
    df = pd.read_csv("Data/clean_train.csv")

    df['Credit_Score'] = df['Credit_Score'].map({
        'Poor': 0,
        'Standard': 1,
        'Good': 2
    })

    target = 'Credit_Score'
    y = df[target]

    X = df.drop(columns=["Customer_ID", "ID",
"Name", "SSN", "Month", target])

    X_train, _, y_train, _ = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    model = LogisticRegression(solver='lbfgs',
max_iter=1000)
    model.fit(X_train, y_train)

    joblib.dump(model,
'Models/benchmark_model.pkl')
```

```
def weights_model_fitter():
    """
    Fit a logistic regression model on the WoE-encoded training data to get feature weights.
    """
    df = pd.read_csv("Data/clean_train.csv")

    X_train, _, y_train, _ = data_splitter(df, target='Credit_Score')

    model = LogisticRegression(solver='lbfgs', max_iter=1000)
    model.fit(X_train, y_train)

    joblib.dump(model, 'Models/weights_model.pkl')


def main():
    benchmark_fitter()
    weights_model_fitter()


if __name__ == "__main__":
    main()
```

credit_model.py

```
import pandas as pd


def get_weights(model, features: list) -> pd.DataFrame:
    """
    Extract feature weights from a fitted model and return them in a DataFrame.

    Parameters:
        model (sklearn estimator): A fitted sklearn model with coef_ attribute.
        features (list): List of feature names corresponding to the model's coefficients.

    Returns:
        pd.DataFrame: DataFrame containing features and their corresponding coefficients,
                      sorted by coefficient value in descending order.
    """
    weights = pd.DataFrame({
        'feature': features,
        'coefficient': model.coef_[2]  # class 2 = Good
    }).sort_values('coefficient', ascending=False).reset_index(drop=True)

    return weights


def get_scores(weights: pd.DataFrame, X_train: pd.DataFrame) -> pd.Series:
    """
    Calculate credit scores based on feature weights and input features.

    Parameters:
        weights (pd.DataFrame): DataFrame containing features and their coefficients.
```

```
        X_train (pd.DataFrame): DataFrame containing
the feature values for each customer.

    Returns:
        pd.Series: Series containing the calculated
credit score for each customer.
    """
    weights = weights.set_index('feature')
    # Ensure feature order matches weights
    X_train = X_train[weights.T.columns]
    scores = X_train.dot(weights['coefficient'])
    return scores


def scale_scores(scores: pd.Series, min_value: float)
-> pd.Series:
    """
    Scale raw logistic regression scores to a fixed
integer range.

    Shifts scores to be non-negative, scales by 100,
then clips
    and normalizes between 0 - 600.

    Parameters:
        scores    (pd.Series) : Raw scores from the
logistic regression.
        min_value (float)     : Minimum value used to
shift scores non-negative.

    Returns:
        scaled (pd.Series): Integer scores constrained
between score_min and score_max.
    """
    score_min = 0
    score_max = 600

    shifted = (scores + abs(min_value)) * 100

    # Normalize to [score_min, score_max]
    scaled = (shifted - shifted.min()) / (shifted.max() -
shifted.min())
    scaled = (scaled * (score_max - score_min) +
score_min).astype(int)

    return scaled.clip(score_min, score_max)


def scores_df(weights: pd.DataFrame, X_train:
pd.DataFrame, y_train: pd.Series, X_test:
pd.DataFrame, y_test: pd.Series) ->
tuple[pd.DataFrame, pd.DataFrame]:
    """
    Calculate and scale credit scores for both training
and testing datasets.

    Parameters:
        weights (pd.DataFrame): DataFrame containing
features and their coefficients.
        X_train (pd.DataFrame): Training features
(WoE-encoded).
        y_train (pd.Series): Training labels.
        X_test  (pd.DataFrame): Testing features
(WoE-encoded).
        y_test  (pd.Series): Testing labels.

    Returns:
```

```
        tuple: A tuple containing two DataFrames with original credit scores and scaled scores for train and test sets.
    """
    # - Train -
    scores = get_scores(weights, X_train)
    min_value = scores.min()  # Save min train value for consistent scaling of test scores
    train_scores = pd.DataFrame({
        'Credit_Score': y_train,
        'Score': scale_scores(scores, min_value)
    }).reset_index(drop=True)

    # - Test -
    scores = get_scores(weights, X_test)
    test_scores = pd.DataFrame({
        'Credit_Score': y_test,
        'Score': scale_scores(scores, min_value)
    }).reset_index(drop=True)

    return train_scores, test_scores


def treshold_classification(train_scores: pd.DataFrame, test_scores: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, list]:
    """
    Classify customers into credit score classes based on defined tresholds.

    Parameters:
        train_scores (pd.DataFrame): DataFrame containing 'Credit_Score' and 'Score' for the training set.
        test_scores  (pd.DataFrame): DataFrame containing 'Credit_Score' and 'Score' for the testing set.

    Returns:
        tuple: A tuple containing the modified train_scores and test_scores DataFrames with an added 'Class' column, and the list of tresholds used for classification.
    """
    # Tresholds Definition (arbitraryly set based on score distribution and class separation)
    t1 = 220  # Poor vs Standard
    t2 = 410  # Standard vs Good
    tresholds = [t1, t2]

    # Classify based on tresholds
    train_scores['Class'] = train_scores['Score'].apply(
        lambda x: 0 if x <= t1 else 1 if x < t2 else 2)
    test_scores['Class'] = test_scores['Score'].apply(
        lambda x: 0 if x <= t1 else 1 if x < t2 else 2)

    return train_scores, test_scores, tresholds
```

visualization.py

```
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import ConfusionMatrixDisplay
```

```
plt.rcParams['figure.figsize'] = (12, 5)
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.5
plt.rcParams['grid.linestyle'] = '--'
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 12

blue_colors = [
    "#D9EDFF",
    "#C1E2FF",
    "#A8D5FF",
    "#8AC6FF",
    "#6BB6FF",
    "#4A9EE5",
    "#2E86DE",
    "#0066C0",
    "#0055A5",
    "#003D82"
]

blue_scale =
mcolors.LinearSegmentedColormap.from_list(
    "blue_scale",
    [blue_colors[0], blue_colors[-1]]
)


def plot_score_distribution(df: pd.DataFrame,
tresholds: list, set: str):
    """
    Plot the distribution of the custom credit score.

    Parameters:
        df (pd.DataFrame): DataFrame containing the
'Score' column with credit scores.
        tresholds (list): List of treshold values used for
classifying credit scores.
        set (str): Name of the dataset (e.g., 'Train' or
'Test') for the plot title.
    """
    plt.figure()
    sns.kdeplot(df['Score'], color=blue_colors[-1],
                fill=True, alpha=0.3, linewidth=2)
    plt.title(f"{set} Distribution of Custom Credit
Score")
    plt.xlabel("Custom Credit Score")
    plt.ylabel("Density")
    plt.show()

    plt.figure()
    color_idx = [1, 5, 9]
    labels = {0: 'Poor', 1: 'Standard', 2: 'Good'}

    # Plot Tresholds
    plt.axvline(x=tresholds[0], color='dimgray',
linestyle='--',
                alpha=1.0, label='Poor/Standard
Tresholds')
    plt.axvline(x=tresholds[1], color='black',
linestyle='--',
                alpha=0.7, label='Standard/Good
Tresholds')

    for class_idx in range(3):
        sns.kdeplot(
            df[df['Credit_Score'] == class_idx]['Score'],
            color=blue_colors[color_idx[class_idx]],
```

```
            fill=True,
            alpha=0.3,
            linewidth=2,
            label=labels[class_idx]
        )

    plt.title(f"{set} Score Distribution per Class")
    plt.xlabel("Score")
    plt.ylabel("Density")
    plt.legend()
    plt.show()


def plot_score_comparison(scores: pd.DataFrame,
set: str):
    """
    Compares the actual credit score distribution with
the classified score distribution.

    Parameters:
        scores (pd.DataFrame): DataFrame containing
'Credit_Score' and 'Score' columns.
        set (str): Name of the dataset (e.g., 'Train' or
'Test') for the plot title.
    """
    real_classes = [
        scores[scores['Credit_Score'] == 0]['Score'],
        scores[scores['Credit_Score'] == 1]['Score'],
        scores[scores['Credit_Score'] == 2]['Score']
    ]

    classified_classes = [
        scores[scores['Class'] == 0]['Score'],
        scores[scores['Class'] == 1]['Score'],
        scores[scores['Class'] == 2]['Score']
    ]

    class_labels = ["Poor", "Standard", "Good"]

    fig, axes = plt.subplots(1, 3, figsize=(18, 5),
sharey=False)

    for i in range(3):

        sns.kdeplot(
            real_classes[i],
            color=blue_colors[2],
            fill=True,
            alpha=0.3,
            linewidth=2,
            label=f"Real {class_labels[i]}",
            ax=axes[i]
        )

        sns.kdeplot(
            classified_classes[i],
            color=blue_colors[-1],
            fill=True,
            alpha=0.3,
            linewidth=2,
            label=f"Classified {class_labels[i]}",
            ax=axes[i]
        )

        axes[i].set_title(f"{class_labels[i]}")
        axes[i].set_xlabel("Score")

        axes[i].legend()
```

```
    axes[0].set_ylabel("Density")
    fig.suptitle(f"{set} Real vs Classified Score Distribution", fontsize=16)

    plt.tight_layout()
    plt.show()


def plot_confusion_matrix(scores: pd.DataFrame, set: str) -> str:
    """
    Plot the confusion matrix for the given true and predicted labels.

    Parameters:
        scores (pd.DataFrame): DataFrame containing 'Credit_Score' (true labels) and 'Class' (predicted labels).
        set (str): Name of the dataset (e.g., 'Train' or 'Test') for the plot title.

    Returns:
        class_report (str): The classification report as a string.
    """
    acc = accuracy_score(scores['Credit_Score'], scores['Class'])
    class_report = classification_report(
        scores['Credit_Score'], scores['Class'], target_names=['Poor', 'Standard', 'Good'])

    fig, ax = plt.subplots()
    disp = ConfusionMatrixDisplay.from_predictions(
        scores['Credit_Score'],
        scores['Class'],
        cmap=blue_scale,
        colorbar=True,
        values_format='d',
        ax=ax
    )

    ax.set_title(f'{set} Confusion Matrix - Accuracy: {acc:.4f}')
    ax.grid(False)

    plt.show()

    return class_report
```

main.py

```
import pandas as pd
import joblib

from utils import get_benchmark, data_splitter
from credit_model import get_weights, scores_df, treshold_classification
from visualizations import plot_score_distribution, plot_score_comparison, plot_confusion_matrix


def main():
    # Load the cleaned training data
    df = pd.read_csv("Data/clean_train.csv")

    # === Get Benchmark ===
    print("\n=== Benchmark Evaluation ===")
```

```
    model = joblib.load('Models/benchmark_model.pkl')
    acc, class_report = get_benchmark(model, df)

    print(f"\nAccuracy: {acc:.4f}")
    print(f"\nBenchmark Classification Report:\n{class_report}")

    # === Feature Selection ===
    X_train, X_test, y_train, y_test = data_splitter(df, target='Credit_Score')

    # === WoE Encoded Data ===
    features = X_train.columns.tolist()

    # === Credit Model ===
    print("\n=== Credit Model Evaluation ===")

    # --- Get feature weights from the model fitted ---
    model = joblib.load('Models/weights_model.pkl')
    weights = get_weights(model, features)
    print(f"\nFeature Weights (sorted by importance):\n{weights}\n")

    # --- Calculate Scores ---
    train_scores, test_scores = scores_df(
        weights, X_train, y_train, X_test, y_test)

    # --- Treshold Classification ---
    train_scores, test_scores, tresholds = treshold_classification(
        train_scores, test_scores)

    # === Visualizations ===
    plot_score_distribution(train_scores, tresholds, 'Train')
    plot_score_distribution(test_scores, tresholds, 'Test')

    plot_score_comparison(train_scores, 'Train')
    plot_score_comparison(test_scores, 'Test')

    train_class_report = plot_confusion_matrix(train_scores, 'Train')
    test_class_report = plot_confusion_matrix(test_scores, 'Test')

    print(f"\nTrain Classification Report:\n{train_class_report}")
    print(f"\nTest Classification Report:\n{test_class_report}")


if __name__ == "__main__":
    main()
```

dashboard.py

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.graph_objects as go

# Configuración de la página
st.set_page_config(
    page_title="Modelo ADA - Scoring de Crédito",
    page_icon="💰",
    layout="centered"
)
```

```
# Diccionario con los pesos de las variables
PESOS = {
    'Credit_Mix_Good': 0.725155,
    'Payment_of_Min_Amount_Yes': 0.385820,
    'Credit_Mix_Standard': 0.349903,
    'High_spent': 0.282668,
    'Num_Credit_Card': 0.103531,
    'Interest_Rate': 0.042818,
    'Delay_from_due_date': 0.019019,
    'Changed_Credit_Limit': 0.015326,
    'Outstanding_Debt': 0.000081
}

# Diccionario con descripciones de las variables en español
DESCRIPCIONES = {
    'Outstanding_Debt': 'Deuda Pendiente',
    'Interest_Rate': 'Tasa de Interés',
    'Delay_from_due_date': 'Días de Retraso',
    'Num_Credit_Card': 'Número de Tarjetas de Crédito',
    'Credit_Mix_Standard': 'Mix de Crédito Estándar',
    'Changed_Credit_Limit': 'Cambio en Límite de Crédito',
    'Credit_Mix_Good': 'Mix de Crédito Bueno',
    'Payment_of_Min_Amount_Yes': 'Pago del Mínimo',
    'High_spent': 'Gasto Alto'
}

# Inicializar el estado de la sesión
if 'page' not in st.session_state:
    st.session_state.page = 'inicio'
if 'user_inputs' not in st.session_state:
    st.session_state.user_inputs = {}


def calcular_score_final(suma_ponderada):

    primer_calculo = (suma_ponderada + 4.26) * 100
    score_final = np.floor((primer_calculo / 538.79) * 600)

    return primer_calculo, score_final


def crear_semaforo_simple(score):

    fig = go.Figure(go.Indicator(
        mode="gauge+number",
        value=score,

        number={
            "font": {"size": 42}
        },

        gauge={
            "axis": {
                "range": [0, 600],
                "tickmode": "array",
                "tickvals": [0, 220, 410, 600],
                "ticktext": ["0", "220", "410", "600"]
            },

            "bar": {"color": "black", "thickness": 0.25},

            "steps": [
```

```
            {"range": [0, 220], "color": "#e74c3c"},
            {"range": [220, 410], "color": "#f1c40f"},
            {"range": [410, 600], "color": "#27ae60"}
        ],

        "threshold": {
            "line": {"color": "black", "width": 6},
            "thickness": 0.8,
            "value": score
        }
    }
))

fig.update_layout(
    height=350,
    margin=dict(l=40, r=40, t=40, b=40),
    paper_bgcolor="rgba(0,0,0,0)"
)

return fig


def pagina_inicio():
    """Página de inicio del modelo ADA"""

    st.markdown("""
        <style>
        .title-container {
            background: linear-gradient(135deg,
#667eea 0%, #764ba2 100%);
            padding: 3rem;
            border-radius: 20px;
            box-shadow: 0 10px 30px rgba(0,0,0,0.2);
            margin-bottom: 2rem;
            text-align: center;
        }
        .main-title {
            color: white;
            font-size: 5rem;
            font-weight: 900;
            margin: 0;
            text-shadow: 3px 3px 6px rgba(0,0,0,0.3);
            letter-spacing: 5px;
        }
        .subtitle {
            color: rgba(255,255,255,0.9);
            font-size: 1.3rem;
            margin-top: 0.5rem;
            font-weight: 300;
            letter-spacing: 2px;
        }
        .card-container {
            background: white;
            padding: 3rem;
            border-radius: 20px;
            box-shadow: 0 10px 40px rgba(0,0,0,0.1);
            text-align: center;
            margin: 2rem 0;
        }
        .stButton > button {
            background: linear-gradient(135deg,
#667eea 0%, #764ba2 100%);
            color: white;
            font-size: 1.3rem;
            padding: 1rem 3rem;
            border-radius: 50px;
            border: none;
            font-weight: bold;
```

```
            letter-spacing: 2px;
            transition: all 0.3s ease;
            width: 100%;
        }
        .stButton > button:hover {
            transform: translateY(-3px);
            box-shadow: 0 15px 30px rgba(102, 126,
234, 0.4);
        }
        </style>
    """, unsafe_allow_html=True)

    st.markdown("""
    <div class="title-container">
        <h1 class="main-title">MODELO ADA</h1>
        <p class="subtitle">Sistema de Scoring
Crediticio Clásico</p>
    </div>
    """, unsafe_allow_html=True)

    if st.button("🚀 COMENZAR", key="start"):
        st.session_state.page = 'formulario'
        st.rerun()


def pagina_formulario():
    """Página del formulario de entrada de datos"""

    st.markdown("""
        <style>
        .form-header {
            background: linear-gradient(135deg,
#667eea 0%, #764ba2 100%);
            padding: 2rem;
            border-radius: 20px;
            margin-bottom: 2rem;
            color: white;
            text-align: center;
        }
        .form-header h2 {
            font-size: 2.2rem;
            margin: 0;
        }
        .form-header p {
            font-size: 1.1rem;
            opacity: 0.9;
            margin: 0.5rem 0 0 0;
        }
        .info-box {
            background-color: #e3f2fd;
            padding: 1rem;
            border-radius: 10px;
            margin-bottom: 2rem;
            text-align: center;
            color: #1976d2;
            font-size: 1rem;
            border-left: 5px solid #1976d2;
        }
        .variable-section {
            background: #f8f9fa;
            padding: 1.5rem;
            border-radius: 15px;
            margin: 1rem 0;
            border-left: 5px solid #667eea;
        }
        .section-title {
            color: #2d3748;
            font-size: 1.5rem;
```

```
        font-weight: bold;
        margin-bottom: 1.5rem;
    }
    .stButton > button {
        background: linear-gradient(135deg,
#667eea 0%, #764ba2 100%);
        color: white;
        font-size: 1.3rem;
        padding: 1rem;
        border-radius: 10px;
        border: none;
        font-weight: bold;
        width: 100%;
    }
    .back-button {
        background: #718096 !important;
    }
    </style>
"""", unsafe_allow_html=True)

col1, col2, col3 = st.columns([1, 2, 1])
with col1:
    if st.button("← VOLVER", key="back"):
        st.session_state.page = 'inicio'
        st.rerun()

with col2:
    st.markdown"""
        <div class="form-header">
            <h2>📊 DATOS DEL SOLICITANTE</h2>
            <p>Complete todos los campos con la
información requerida</p>
        </div>
    """, unsafe_allow_html=True)

st.markdown"""
    <div class="info-box">
        ℹ️ Todos los campos aceptan valores
positivos y negativos
    </div>
"""", unsafe_allow_html=True)

col1, col2 = st.columns(2)

with col1:
    with st.container():
        st.markdown('<div class="variable-section">',
                    unsafe_allow_html=True)
        st.markdown(
            '<p class="section-title">📈 VARIABLES
FINANCIERAS</p>', unsafe_allow_html=True)

        outstanding_debt = st.number_input(
            "💰 Deuda Pendiente",
            value=0.0,
            step=1000.0,
            format="%.2f",
            key="debt"
        )

        interest_rate = st.number_input(
            "📊 Tasa de Interés (%)",
            value=0.0,
            step=0.5,
            format="%.2f",
            key="rate"
        )
```

```
        delay_days = st.number_input(
            "⏰ Días de Retraso",
            value=0,
            step=1,
            key="delay"
        )

        num_cards = st.number_input(
            "💳 Número de Tarjetas",
            value=0,
            step=1,
            key="cards"
        )

        high_spent = st.number_input(
            "💸 Gasto Alto",
            value=0.0,
            step=1000.0,
            format="%.2f",
            key="spent"
        )
        st.markdown('</div>',
unsafe_allow_html=True)

    with col2:
        with st.container():
        st.markdown('<div class="variable-section">',
                    unsafe_allow_html=True)
        st.markdown(
            '<p class="section-title">📋 VARIABLES
DE COMPORTAMIENTO</p>',
unsafe_allow_html=True)

        credit_mix_std = st.number_input(
            "🔄 Mix Crédito Estándar",
            value=0.0,
            step=0.1,
            format="%.2f",
            key="mix_std"
        )

        changed_limit = st.number_input(
            "📈 Cambio en Límite",
            value=0.0,
            step=1000.0,
            format="%.2f",
            key="limit_change"
        )

        credit_mix_good = st.number_input(
            "✨ Mix Crédito Bueno",
            value=0.0,
            step=0.1,
            format="%.2f",
            key="mix_good"
        )

        payment_min = st.number_input(
            "💶 Pago Mínimo",
            value=0.0,
            step=0.1,
            format="%.2f",
            key="payment"
        )
        st.markdown('</div>',
unsafe_allow_html=True)
```

```
    if st.button("📊 CALCULAR SCORE",
key="calculate"):
        st.session_state.user_inputs = {
            'Outstanding_Debt': outstanding_debt,
            'Interest_Rate': interest_rate,
            'Delay_from_due_date': delay_days,
            'Num_Credit_Card': num_cards,
            'Credit_Mix_Standard': credit_mix_std,
            'Changed_Credit_Limit': changed_limit,
            'Credit_Mix_Good': credit_mix_good,
            'Payment_of_Min_Amount_Yes':
payment_min,
            'High_spent': high_spent
        }
        st.session_state.page = 'resultado'
        st.rerun()


def pagina_resultado():
    """Página de resultados con semáforo simple"""

    st.markdown("""
        <style>
        .result-header {
            background: linear-gradient(135deg,
#667eea 0%, #764ba2 100%);
            padding: 2rem;
            border-radius: 20px;
            margin-bottom: 2rem;
            color: white;
            text-align: center;
        }
        .result-header h2 {
            font-size: 2.2rem;
            margin: 0;
        }
        .result-header p {
            font-size: 1.1rem;
            opacity: 0.9;
            margin: 0.5rem 0 0 0;
        }
        .calc-box {
            background: #f8f9fa;
            padding: 1.5rem;
            border-radius: 15px;
            margin: 1rem 0;
            border-left: 5px solid #667eea;
        }
        .calc-title {
            color: #2d3748;
            font-size: 1.2rem;
            font-weight: bold;
            margin-bottom: 1rem;
        }
        .calc-value {
            font-size: 1.5rem;
            font-weight: bold;
            color: #667eea;
        }
        .stButton > button {
            background: linear-gradient(135deg,
#667eea 0%, #764ba2 100%);
            color: white;
            font-size: 1.2rem;
            padding: 0.8rem;
            border-radius: 10px;
            border: none;
            font-weight: bold;
```

```
        width: 100%;
    }
    .semaforo-container {
        background: white;
        padding: 2rem;
        border-radius: 20px;
        box-shadow: 0 5px 20px rgba(0,0,0,0.05);
        margin: 2rem 0;
    }
    </style>
"""", unsafe_allow_html=True)

# Calcular la suma ponderada
suma_ponderada = 0
for var_name, valor in st.session_state.user_inputs.items():
    if var_name in PESOS:
        suma_ponderada += valor * PESOS[var_name]

# Calcular scores
primer_calculo, score_final = calcular_score_final(suma_ponderada)

# Determinar color y texto según el score
if score_final < 220:
    color_score = "#FF4B4B"
    nivel_riesgo = "ALTO"
    emoji = "🔴"
elif score_final < 410:
    color_score = "#FFD700"
    nivel_riesgo = "MEDIO"
    emoji = "🟡"
else:
    color_score = "#4CAF50"
    nivel_riesgo = "BAJO"
    emoji = "🟢"

# Header
col1, col2, col3 = st.columns([1, 2, 1])
with col1:
    if st.button("← NUEVA EVALUACIÓN", key="new"):
        st.session_state.page = 'formulario'
        st.rerun()

with col2:
    st.markdown""""
        <div class="result-header">
            <h2>📊 RESULTADO DEL SCORING</h2>
            <p>Evaluación crediticia completada</p>
        </div>
    """", unsafe_allow_html=True)

# Score y nivel de riesgo ARRIBA
st.markdown(f""""
<div style="text-align: center; margin: 0.1rem 0;">
    <span style="font-size: 3rem; font-weight: 900; color: {color_score};">{score_final:.0f}</span>
    <span style="font-size: 2rem; margin-left: 1rem;">{emoji}</span>
    <div style="font-size: 1.5rem; color: #666; margin-top: 0.5rem;">
        Nivel de Riesgo: <span style="color: {color_score}; font-weight: bold;">{nivel_riesgo}</span>
    </div>
```

```
    </div>
    """, unsafe_allow_html=True)

    # Gráfica abajo
    st.markdown('<div class="semaforo-container">',
unsafe_allow_html=True)
    fig = crear_semaforo_simple(score_final)
    st.plotly_chart(fig, use_container_width=True)
    st.markdown('</div>', unsafe_allow_html=True)

    # Cálculos en columnas
    col1, col2, col3 = st.columns(3)

    with col1:
        st.markdown(f"""
            <div class="calc-box">
                <div class="calc-title">📝 SUMA
PONDERADA</div>
                <div class="calc-
value">{suma_ponderada:.4f}</div>
            </div>
        """, unsafe_allow_html=True)

    with col2:
        st.markdown(f"""
            <div class="calc-box">
                <div class="calc-title">📊 PRIMER
CÁLCULO</div>
                <div class="calc-
value">{primer_calculo:.2f}</div>
                <div style="font-size:0.9rem;
color:#666;">({suma_ponderada:.4f} + 4.26) ×
100</div>
            </div>
        """, unsafe_allow_html=True)

    with col3:
        st.markdown(f"""
            <div class="calc-box">
                <div class="calc-title">🎯 SCORE
FINAL</div>
                <div class="calc-value"
style="color:{color_score};">{score_final:.0f}</div>
                <div style="font-size:0.9rem;
color:#666;">[({primer_calculo:.2f}/538.79)×600]</di
v>
            </div>
        """, unsafe_allow_html=True)

    # Decisión crediticia
    st.markdown("### 📋 DECISIÓN CREDITICIA")

    if score_final < 220:
        st.error(f"""
            ### 🔴 RIESGO ALTO
            - **Score:** {score_final:.0f} puntos
            - **Decisión:** Rechazar solicitud
            - **Motivo:** Alto riesgo de incumplimiento
            - **Recomendación:** No otorgar crédito
        """)
    elif score_final < 410:
        st.warning(f"""
            ### 🟡 RIESGO MEDIO
            - **Score:** {score_final:.0f} puntos
            - **Decisión:** Evaluar con detenimiento
            - **Motivo:** Riesgo moderado
            - **Recomendación:** Solicitar garantías
adicionales
```

```
        """)
    else:
        st.success(f"""
            ### 🟢 RIESGO BAJO
            - **Score:** {score_final:.0f} puntos
            - **Decisión:** Aprobar solicitud
            - **Motivo:** Excelente perfil crediticio
            - **Recomendación:** Aprobación
automática
        """)


# Control de navegación
if st.session_state.page == 'inicio':
    pagina_inicio()
elif st.session_state.page == 'formulario':
    pagina_formulario()
elif st.session_state.page == 'resultado':
    pagina_resultado()
```